

NEURAL RENDERING

Applications of Deep Learning – Spring 2024

Rubén Martínez-Cantín, Julio Marco

Master in Robotics, Graphics and Computer Vision – Universidad de Zaragoza

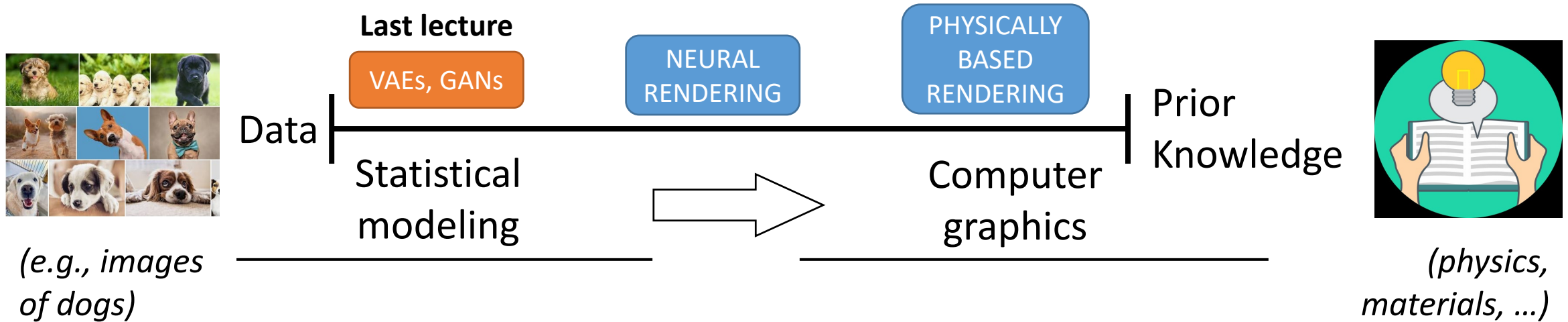
Slide credits

Part of these slides are based on the following content:

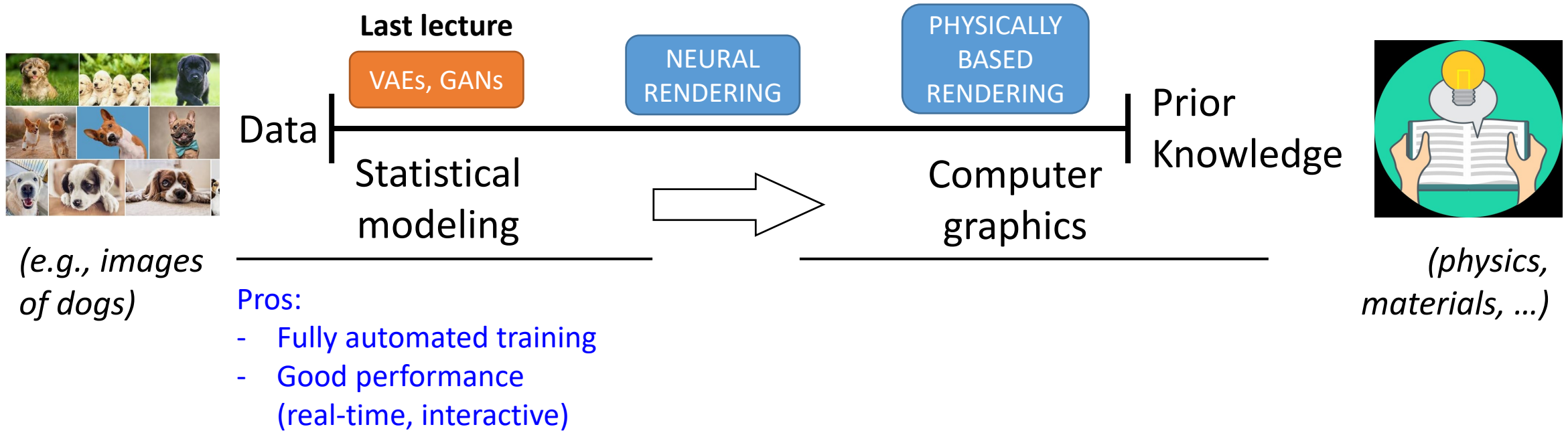
- [State-of-the-Art on Neural Rendering](#), Tewari et al., Eurographics 2020
- [Neural Rendering](#), CVPR 2020 Tutorial
- [Neural Rendering](#), ADL4CV, *Technical University of Munich*

Please refer to the previous links for further information about today's topic.

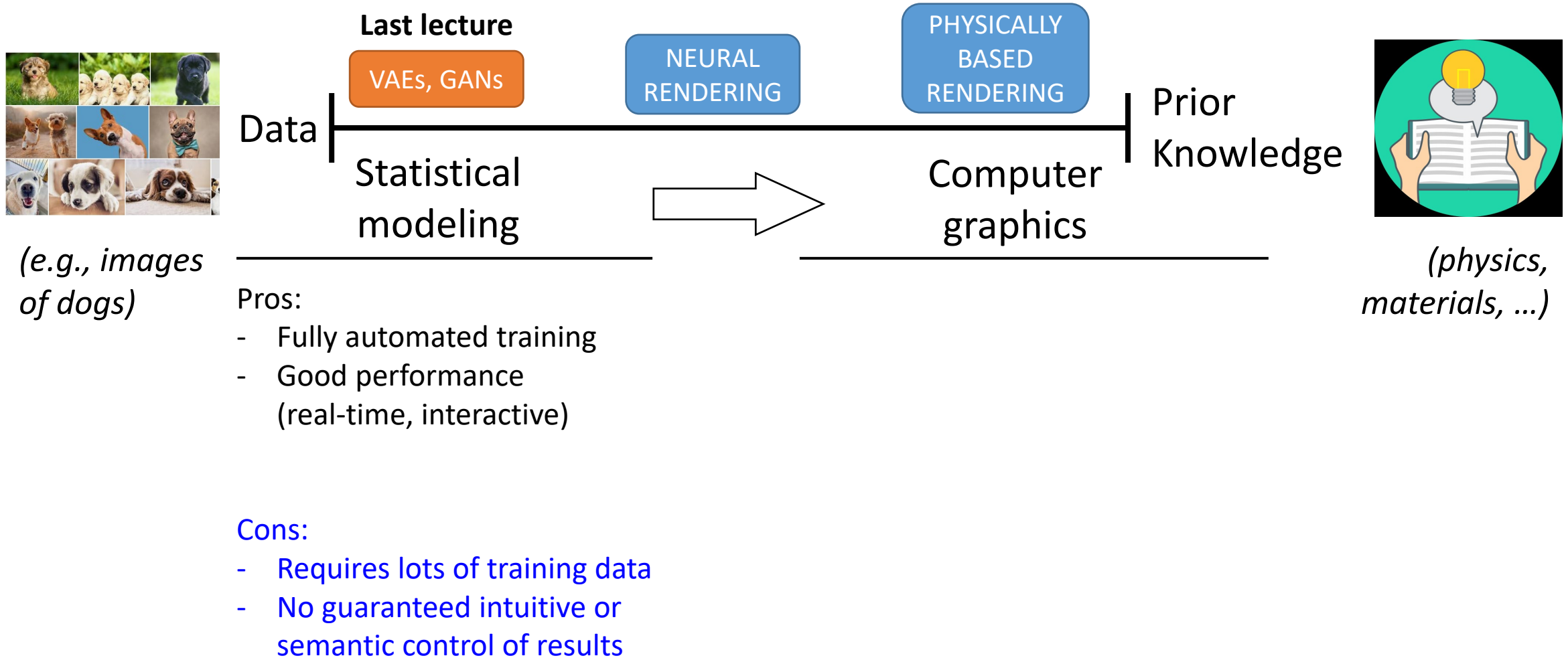
Content generation



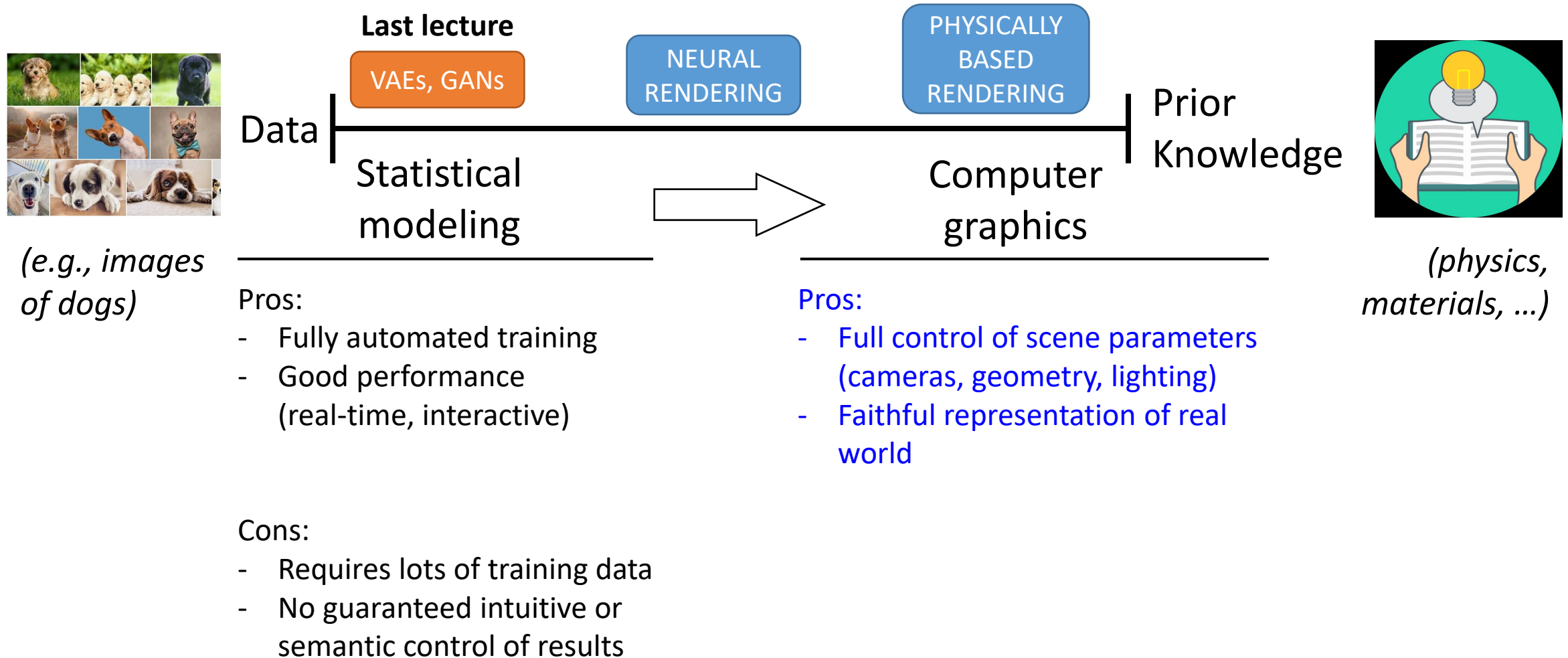
Content generation



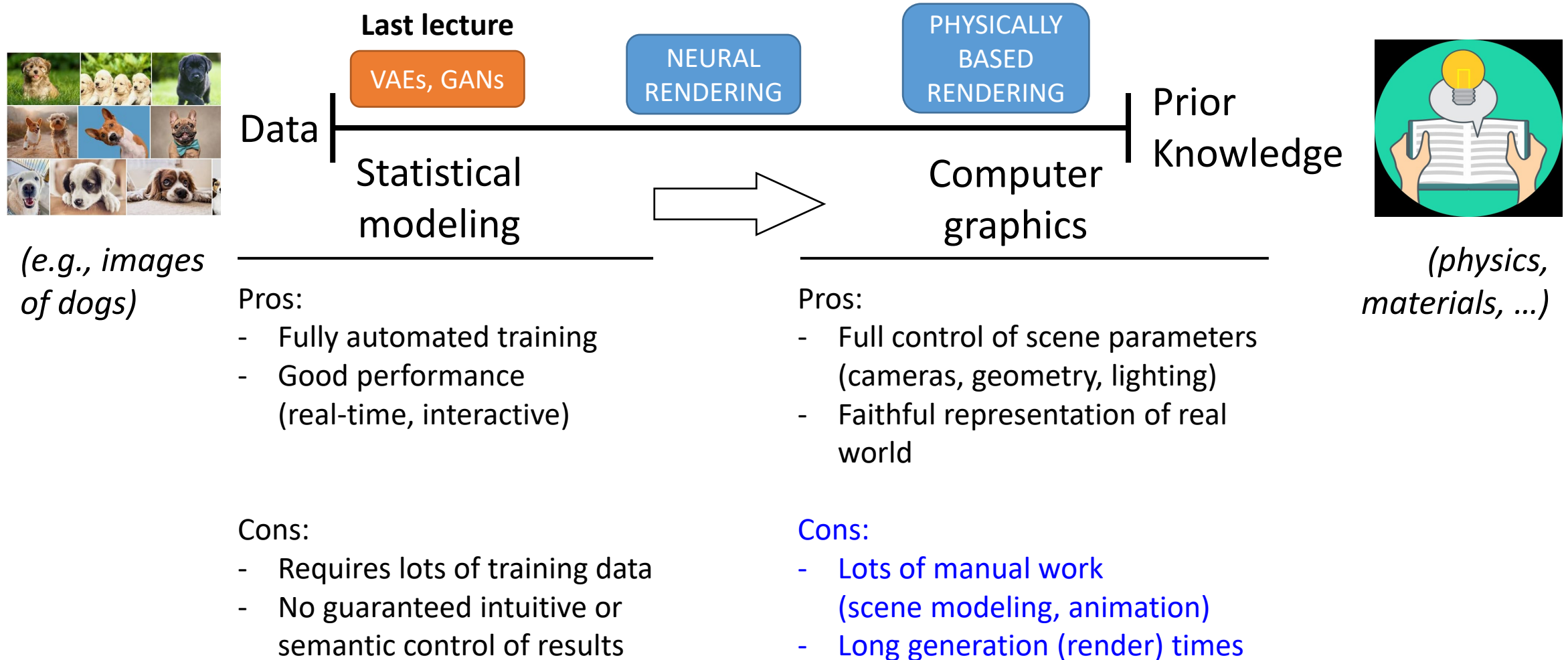
Content generation



Content generation



Content generation



Content generation



*(e.g., images
of dogs)*

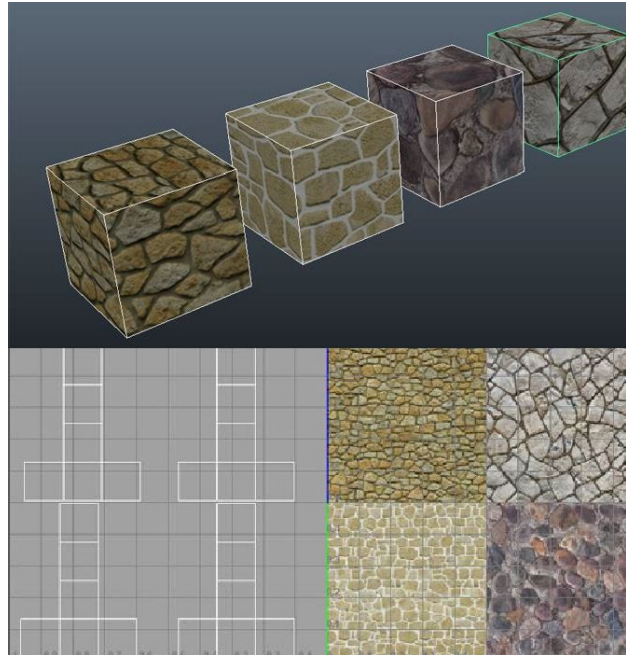
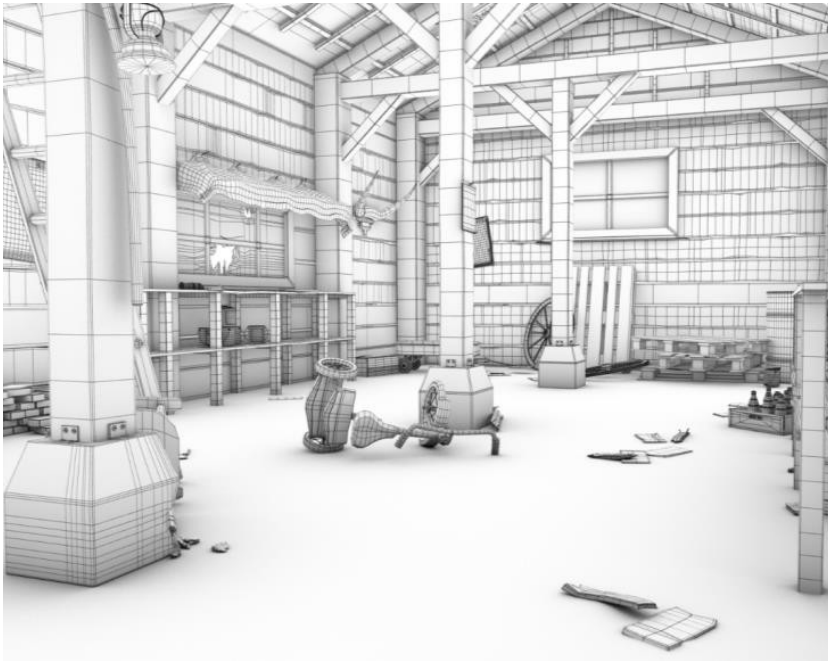


*(physics,
materials, ...)*

Why neural rendering?

Generating content...

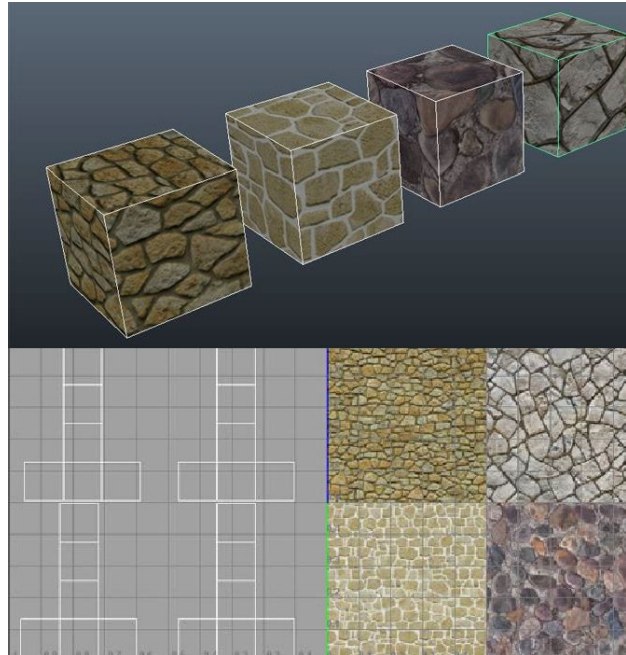
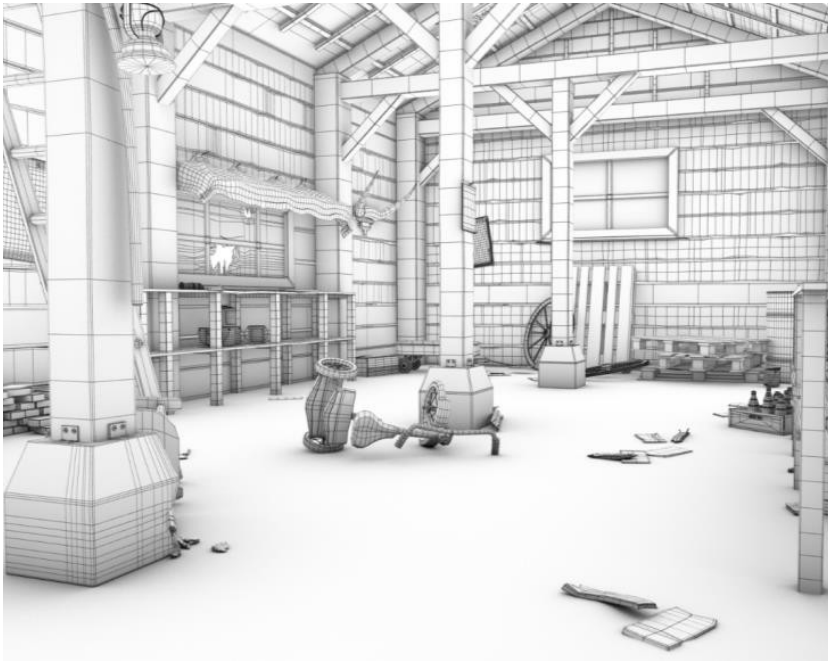
- For rendering → Photorealistic imagery



Why neural rendering?

Generating content...

- For rendering → Photorealistic imagery, **efficiently!**



Why neural rendering?

Generating content...

- For rendering → Photorealistic imagery, **efficiently!**

Transparency



Glossy



Thin Structures



Skin



Cloth



Faces

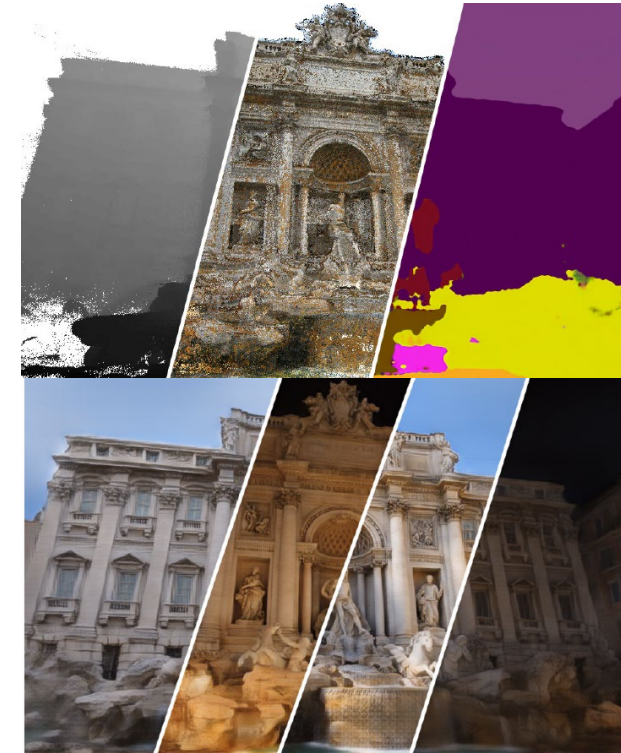


[Lombardi et al. 2019]

Why neural rendering?

Generating content...

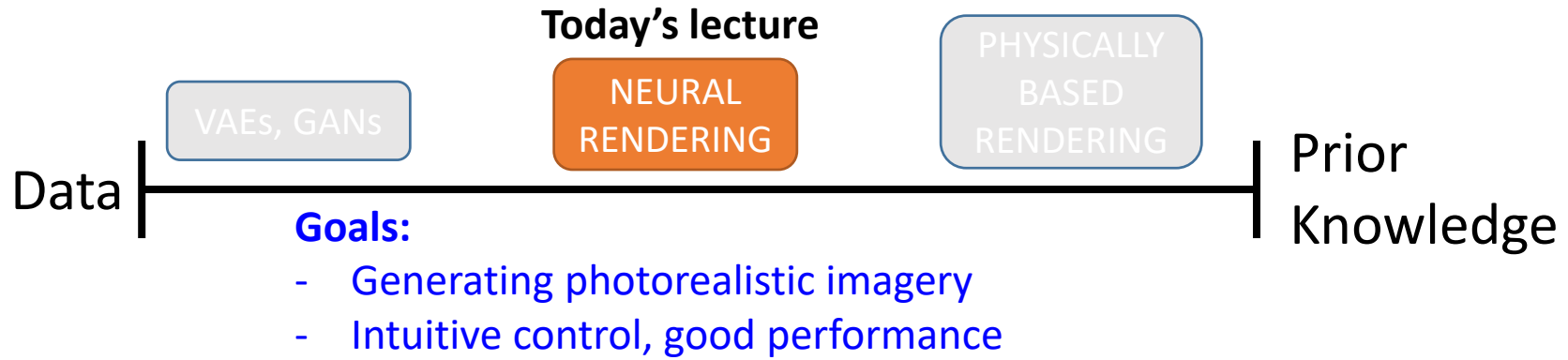
- For rendering → Photorealistic imagery, **intuitively!**



Content generation



(e.g., images
of dogs)



(physics,
materials, ...)

Content generation

Today's lecture



(e.g., images of dogs)



Goals:

- Generating photorealistic imagery
- Intuitive control, good performance

Controllable input parameters:

- Task-motivated:
 - ➔ We don't want to control everything
- Physics-driven:
 - ➔ Predictable effect on output



(physics, materials, ...)

Content generation



(e.g., images of dogs)



Goals:

- Generating photorealistic imagery
- Intuitive control, good performance

Controllable input parameters:

- Task-motivated:
 - ➔ We don't want to control everything
- Physics-driven:
 - ➔ Predictable effect on output

Neural encoding of scenes:

- Statistical approximation of real world
- Abstraction of high-dimensional space (materials? geometry? lighting?)
- Compliant with physical behavior
 - Integrate physics models ➔ Differentiability!!



(physics, materials, ...)

Content generation



(e.g., images of dogs)



Goals:

- Generating photorealistic imagery
- Intuitive control, good performance

Controllable input parameters:

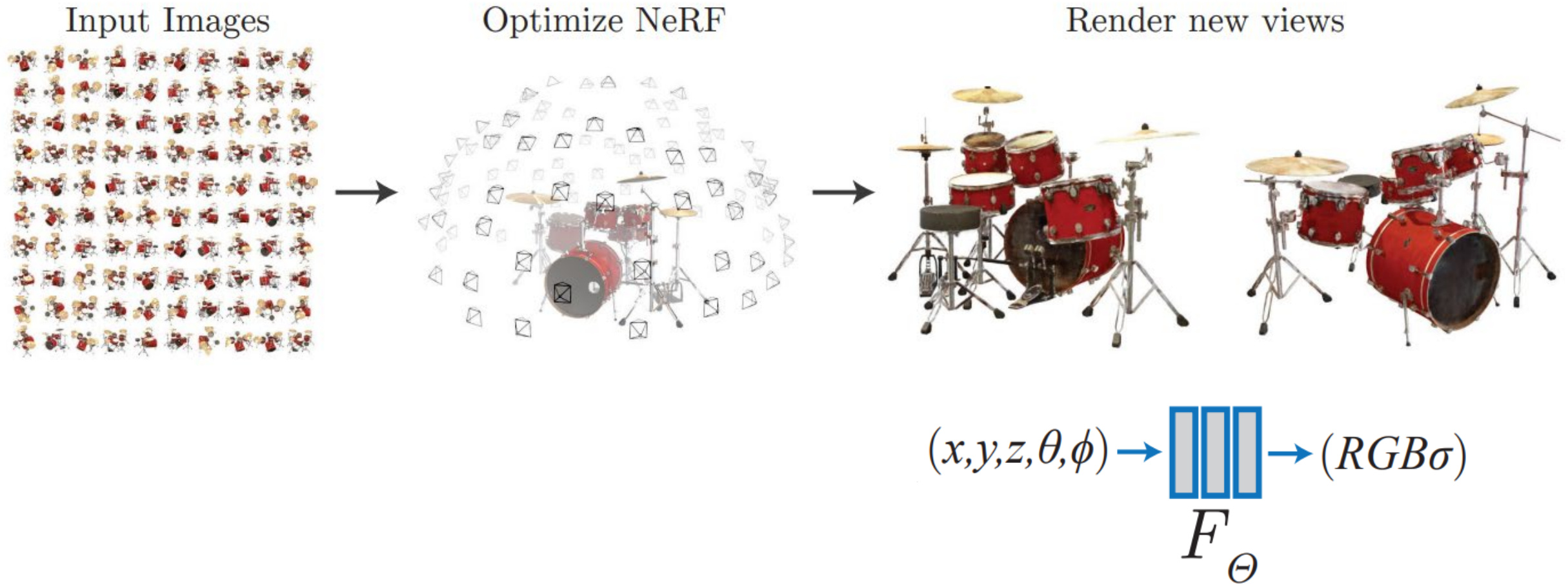
- Task-motivated:
 - ➔ We don't want to control everything
- Physics-driven:
 - ➔ Predictable effect on output

- Rendering equation
- Radiative transfer
- Material models
- Lighting models
- Camera models
- ...

Neural encoding of scenes:

- Statistical approximation of real world
 - Abstraction of high-dimensional space (materials? geometry? lighting?)
 - Compliant with physical behavior
- Integrate physics models ➔ Differentiability!!

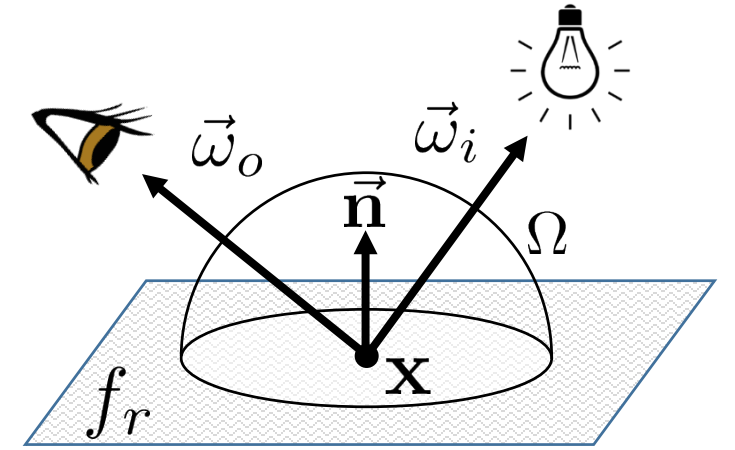
Neural rendering example: Neural Radiance Fields



Rendering brush-up

- The rendering equation:

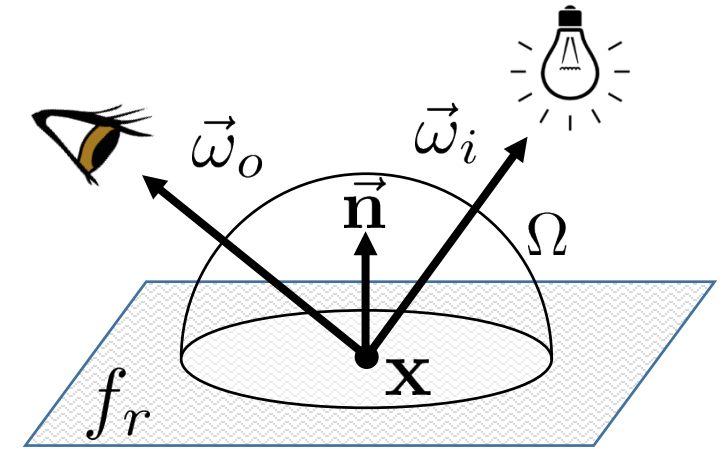
$$L_o(\mathbf{x}, \vec{\omega}_o) = \underbrace{L_e(\mathbf{x}, \vec{\omega}_o)}_{\text{EMISSION}} + \int_{\Omega} \underbrace{L_i(\mathbf{x}, \vec{\omega}_i)}_{\text{INCOMING LIGHT}} \underbrace{f_r(\mathbf{x}, \vec{\omega}_i, \vec{\omega}_o)}_{\text{BRDF}} \underbrace{(\vec{\mathbf{n}} \cdot \vec{\omega}_i)}_{\text{GEOMETRIC TERM}} d\vec{\omega}_i$$



Rendering brush-up

- The rendering equation:

$$L_o(\mathbf{x}, \vec{\omega}_o) = \underbrace{L_e(\mathbf{x}, \vec{\omega}_o)}_{\text{EMISSION}} + \int_{\Omega} \underbrace{L_i(\mathbf{x}, \vec{\omega}_i)}_{\text{INCOMING LIGHT}} \underbrace{f_r(\mathbf{x}, \vec{\omega}_i, \vec{\omega}_o)}_{\text{BRDF}} \underbrace{(\vec{\mathbf{n}} \cdot \vec{\omega}_i)}_{\text{GEOMETRIC TERM}} d\vec{\omega}_i$$



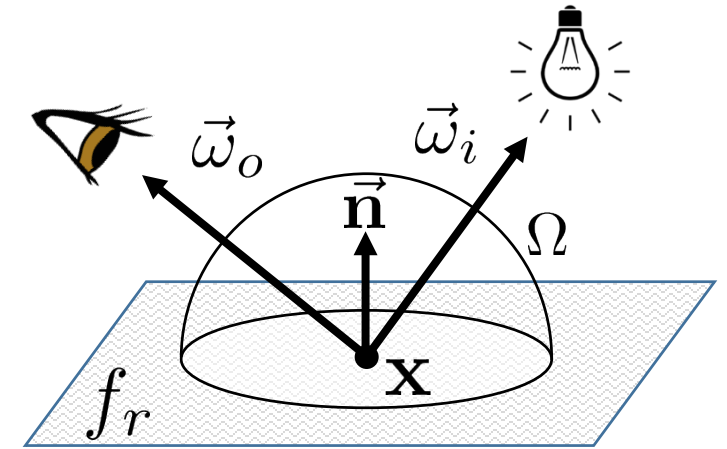
At a single location $\mathbf{x}$:

- **Viewpoint** = Outgoing light direction $\vec{\omega}_o$ (2D angular coordinates)
- **Direct & indirect light** = Incoming light direction $\vec{\omega}_i$ (2D angular coordinates)
- **Surface appearance, reflectance** = Multi-dimensional function f_r (4D in/out coords. + specific BRDF parameters)
- **Scene elements** = Surrounding environment $\Omega \rightarrow$ **Thousands or millions of variables (meshes, media...)**

Rendering brush-up

- The rendering equation:

$$L_o(\mathbf{x}, \vec{\omega}_o) = \underbrace{L_e(\mathbf{x}, \vec{\omega}_o)}_{\text{EMISSION}} + \int_{\Omega} \underbrace{L_i(\mathbf{x}, \vec{\omega}_i)}_{\text{INCOMING LIGHT}} \underbrace{f_r(\mathbf{x}, \vec{\omega}_i, \vec{\omega}_o)}_{\text{BRDF}} \underbrace{(\vec{\mathbf{n}} \cdot \vec{\omega}_i)}_{\text{GEOMETRIC TERM}} d\vec{\omega}_i$$



Physically-based rendering pipeline

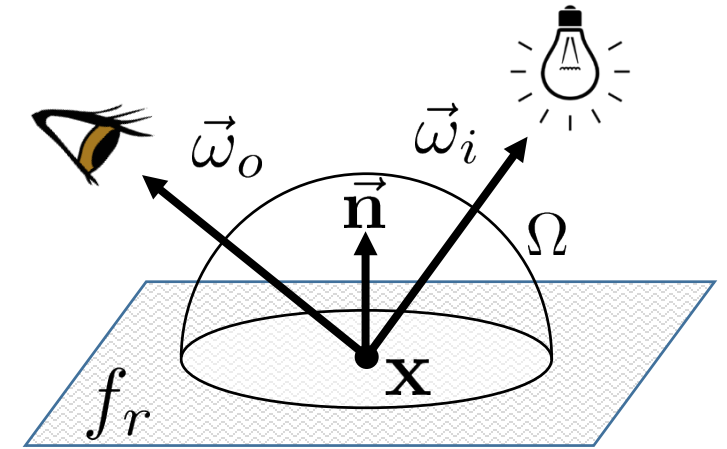
Input information:

- Lighting
- Camera
- Geometry
- Materials
- Participating media
- ...

Rendering brush-up

- The rendering equation:

$$L_o(\mathbf{x}, \vec{\omega}_o) = \underbrace{L_e(\mathbf{x}, \vec{\omega}_o)}_{\text{EMISSION}} + \int_{\Omega} \underbrace{L_i(\mathbf{x}, \vec{\omega}_i)}_{\text{INCOMING LIGHT}} \underbrace{f_r(\mathbf{x}, \vec{\omega}_i, \vec{\omega}_o)}_{\text{BRDF}} \underbrace{(\vec{\mathbf{n}} \cdot \vec{\omega}_i)}_{\text{GEOMETRIC TERM}} d\vec{\omega}_i$$



Physically-based rendering pipeline

Input information:

- Lighting
- Camera
- Geometry
- Materials
- Participating media
- ...

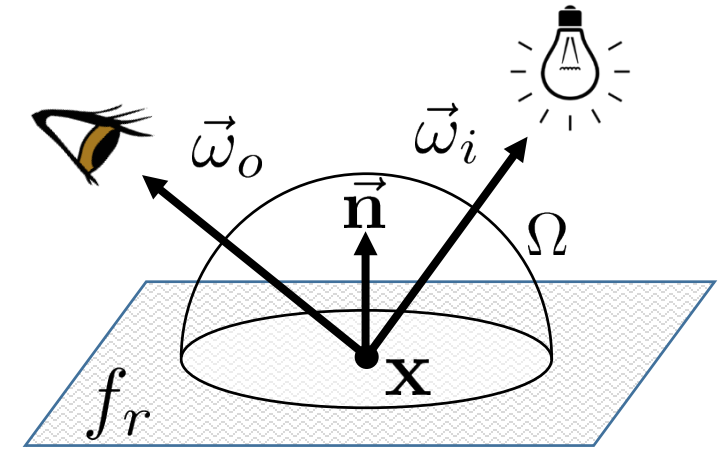


Rendering engine
Physics-driven models taking scene parameters (input).

Rendering brush-up

- The rendering equation:

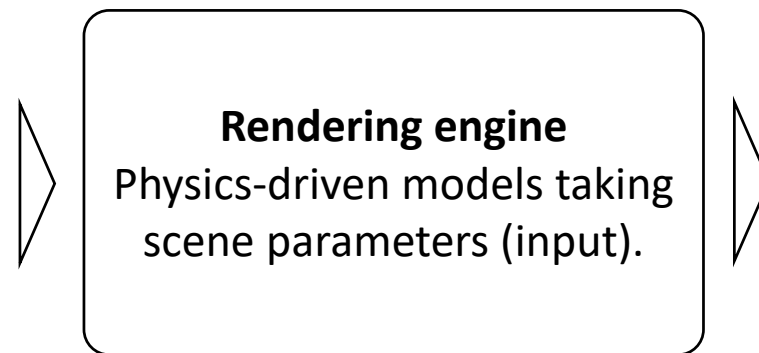
$$L_o(\mathbf{x}, \vec{\omega}_o) = \underbrace{L_e(\mathbf{x}, \vec{\omega}_o)}_{\text{EMISSION}} + \int_{\Omega} \underbrace{L_i(\mathbf{x}, \vec{\omega}_i)}_{\text{INCOMING LIGHT}} \underbrace{f_r(\mathbf{x}, \vec{\omega}_i, \vec{\omega}_o)}_{\text{BRDF}} \underbrace{(\vec{\mathbf{n}} \cdot \vec{\omega}_i)}_{\text{GEOMETRIC TERM}} d\vec{\omega}_i$$



Physically-based rendering pipeline

Input information:

- Lighting
- Camera
- Geometry
- Materials
- Participating media
- ...

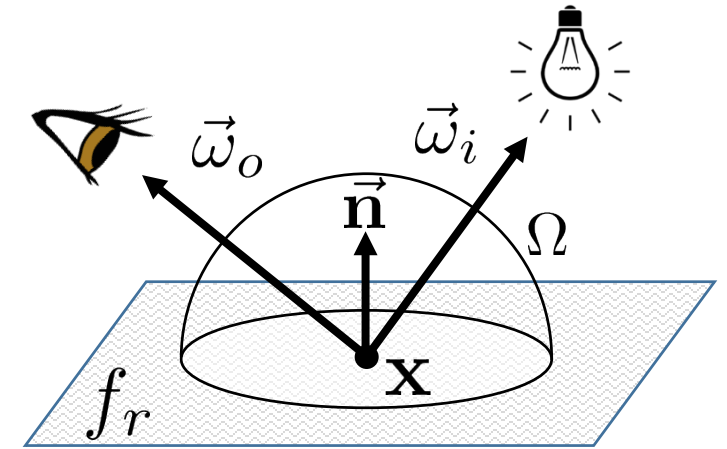


Nice-looking render

Rendering → Neural rendering

- The rendering equation:

$$L_o(\mathbf{x}, \vec{\omega}_o) = \underbrace{L_e(\mathbf{x}, \vec{\omega}_o)}_{\text{EMISSION}} + \int_{\Omega} \underbrace{L_i(\mathbf{x}, \vec{\omega}_i)}_{\text{INCOMING LIGHT}} \underbrace{f_r(\mathbf{x}, \vec{\omega}_i, \vec{\omega}_o)}_{\text{BRDF}} \underbrace{(\vec{\mathbf{n}} \cdot \vec{\omega}_i)}_{\text{GEOMETRIC TERM}} d\vec{\omega}_i$$



NEURAL RENDERING pipeline

Input information:

- Lighting
- Camera
- Geometry
- Materials
- Participating media
- ...

Rendering engine
Physics-driven models taking scene parameters (input).

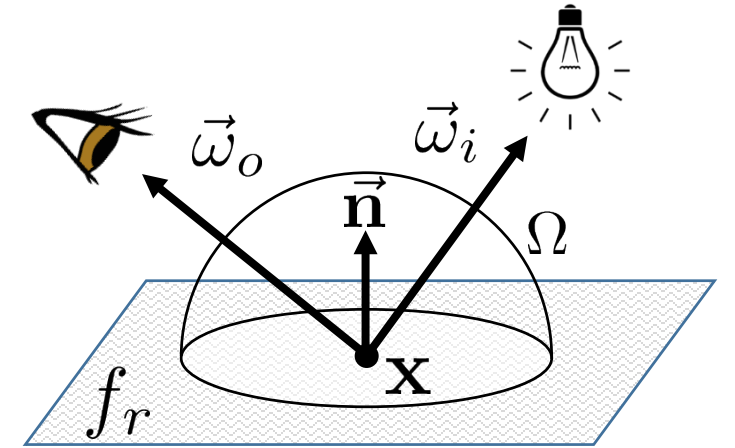


Nice-looking render

Rendering → Neural rendering

- The rendering equation:

$$L_o(\mathbf{x}, \vec{\omega}_o) = \underbrace{L_e(\mathbf{x}, \vec{\omega}_o)}_{\text{EMISSION}} + \int_{\Omega} \underbrace{L_i(\mathbf{x}, \vec{\omega}_i)}_{\text{INCOMING LIGHT}} \underbrace{f_r(\mathbf{x}, \vec{\omega}_i, \vec{\omega}_o)}_{\text{BRDF}} \underbrace{(\vec{\mathbf{n}} \cdot \vec{\omega}_i)}_{\text{GEOMETRIC TERM}} d\vec{\omega}_i$$



NEURAL RENDERING pipeline

Input information:

- Lighting
- AND/OR
- Camera
- OR
- Materials
- OR
- ...any other parameters

Neural architecture

- Encodes scene description (anything that's not an input).
- Approximates physically-based rendering models.

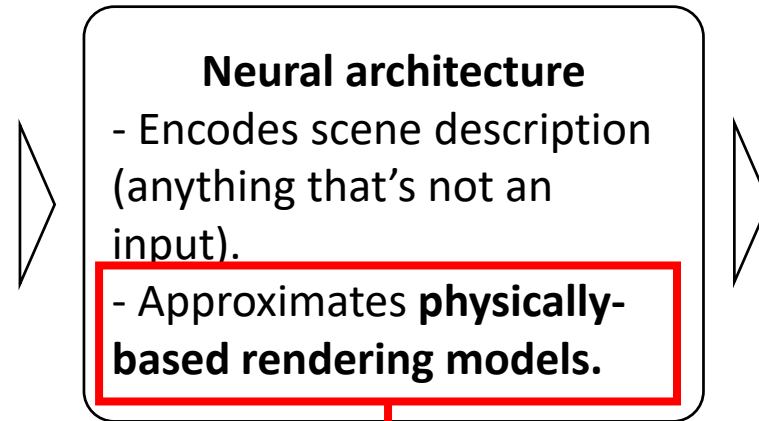


Hopefully a nice-looking render

Neural rendering

Input information:

- Lighting
- AND/OR
- Camera
- OR
- Materials
- OR
- ...any other parameters



Physically-based rendering
mimics **real-world behavior**

Synthesize new imagery from **both
synthetic and real datasets**

Neural rendering

Input information:

- Lighting
- AND/OR
- Camera
- OR
- Materials
- OR
- ...any other parameters

Neural architecture

- Encodes scene description (anything that's not an input).
- Approximates **physically-based rendering models**.



Training

Ground truth information:

- Target images → **Rendered or captured**
- Input parameters → Known/estimated a priori
- Loss between output and GT images

Testing

New information:

- Input parameters (e.g. new viewpoint, light position)
- Generate new image

Neural rendering: Teaser example

Input information:

- Camera viewpoint



Labeled data:

Images + camera viewpoint



Training

Ground truth information:

- Target images
- Input parameters: Camera viewpoint
- Loss between output and GT images

Neural rendering: Teaser example

Input information:

- Camera viewpoint



New image

Testing

New information:

- Input parameters: New viewpoints
- Generate new image not seen before

Neural rendering

Input information:

- Lighting
- AND/OR
- Camera
- OR
- Materials
- OR
- ...any other parameters

Neural architecture

- Encodes scene description (anything that's not an input).
- Approximates **physically-based rendering models**.



CONTROLLABILITY

What parameters can we control to generate content? Implicit or explicit control?

Neural rendering

Input information:

- Lighting
- AND/OR
- Camera
- OR
- Materials
- OR
- ...any other parameters

Neural architecture

- Encodes scene description (anything that's not an input).
- Approximates **physically-based rendering models**.



CONTROLLABILITY

What parameters can we control to generate content? Implicit or explicit control?

IMPLICIT vs. EXPLICIT modeling

What elements are explicitly defined from physics or are implicitly learned from data?

Neural rendering

Input information:

- Lighting
- AND/OR
- Camera
- OR
- Materials
- OR
- ...any other parameters

Neural architecture

- Encodes scene description (anything that's not an input).
- Approximates **physically-based rendering models**.



CONTROLLABILITY

What parameters can we control to generate content? Implicit or explicit control?

IMPLICIT vs. EXPLICIT modeling

What elements are explicitly defined from physics or are implicitly learned from data?

APPLICATION SCOPE

What is our target application? Single or multiple outputs?

Neural rendering

Input information:

- Lighting
- AND/OR
- Camera
- OR
- Materials
- OR
- ...any other parameters

Neural architecture

- Encodes scene description (anything that's not an input).
- Approximates **physically-based rendering models**.



CONTROLLABILITY

What parameters can we control to generate content? Implicit or explicit control?

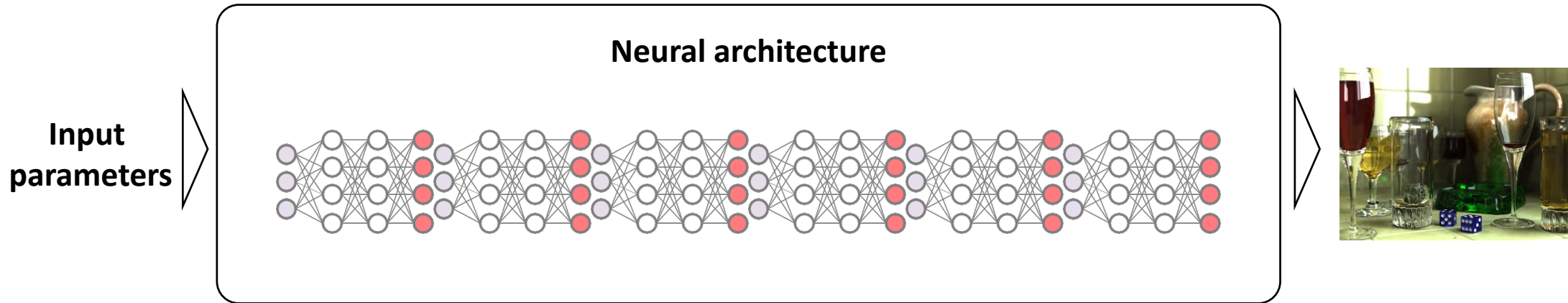
IMPLICIT vs. EXPLICIT modeling

What elements are explicitly defined from physics or are implicitly learned from data?

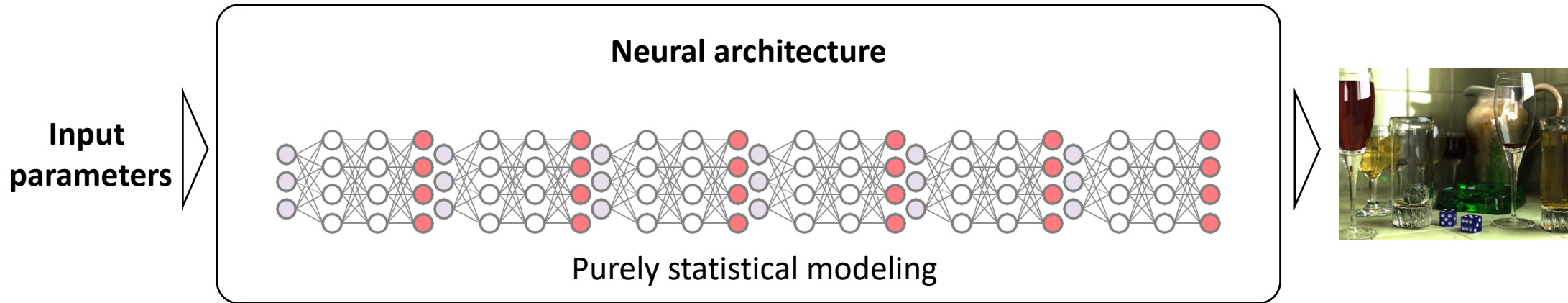
APPLICATION SCOPE

What is our target application? Single or multiple outputs?

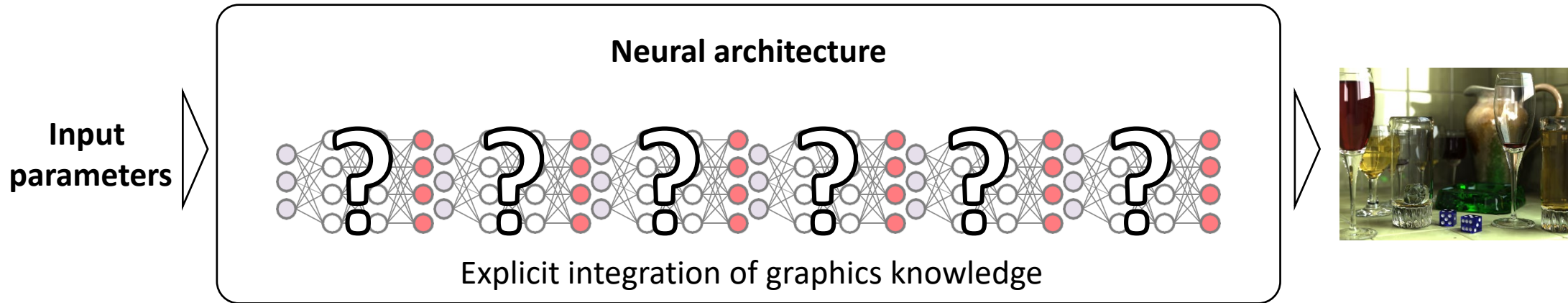
Neural rendering: Integrating CG modules



Neural rendering: Integrating CG modules



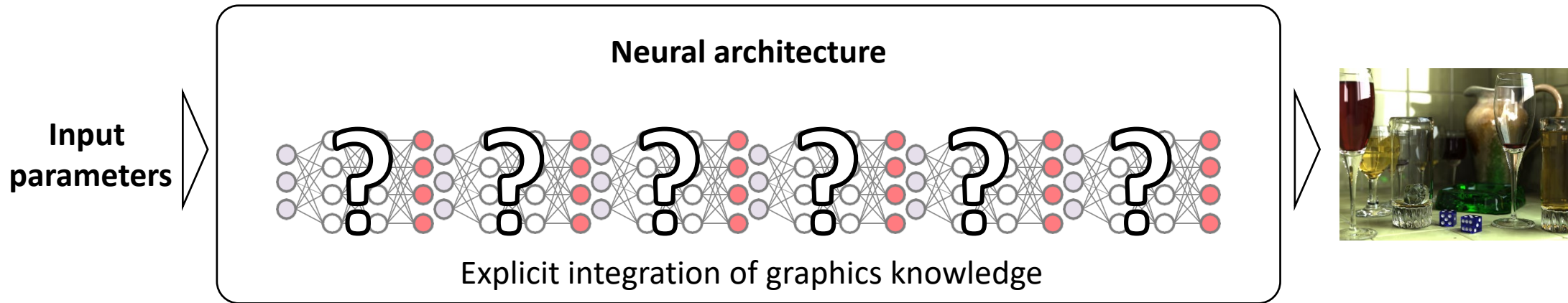
Neural rendering: Integrating CG modules



Explicit integration of scene elements and models:

-
-
-
-
-

Neural rendering: Integrating CG modules

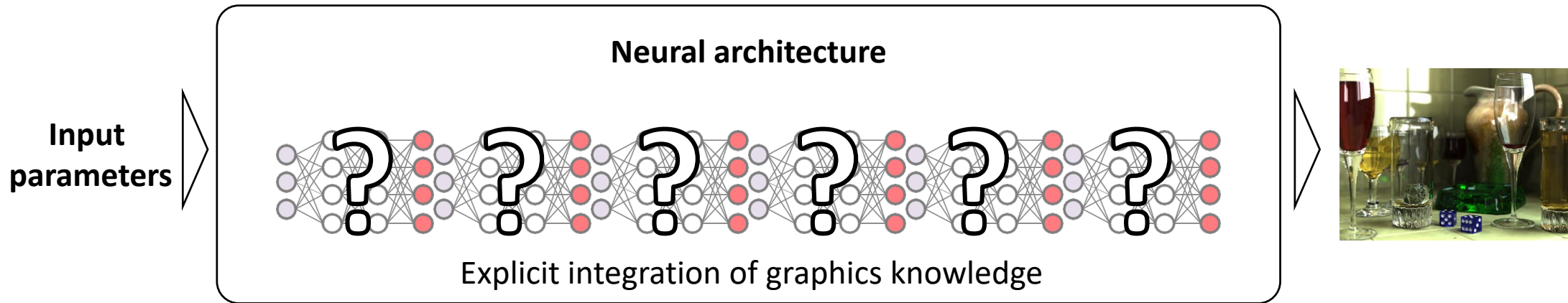


Explicit integration of scene elements and models:

- Geometry: Triangular meshes, primitives (cubes, planes, spheres), voxels, point clouds

-
-
-
-

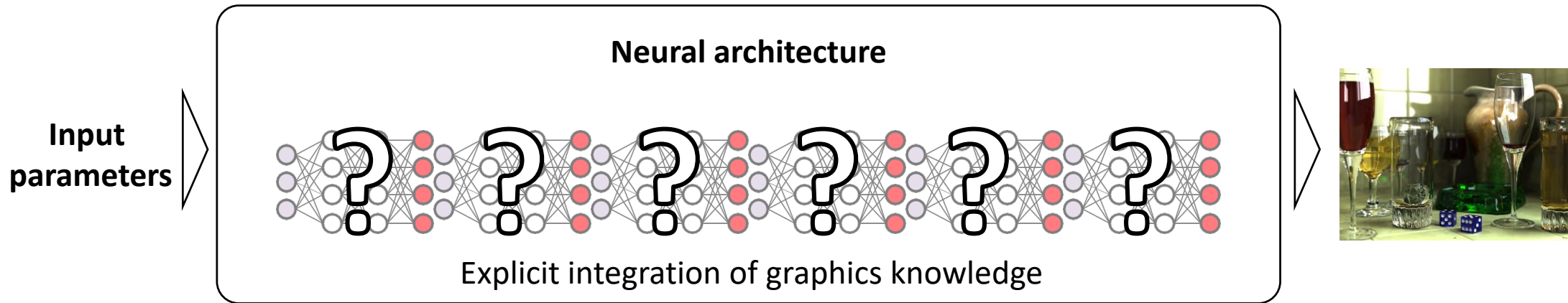
Neural rendering: Integrating CG modules



Explicit integration of scene elements and models:

- Geometry: Triangular meshes, primitives (cubes, planes, spheres), voxels, point clouds
- Lights: Parametric primitives (point, area, sphere), environment maps, triangular meshes
-
-
-

Neural rendering: Integrating CG modules

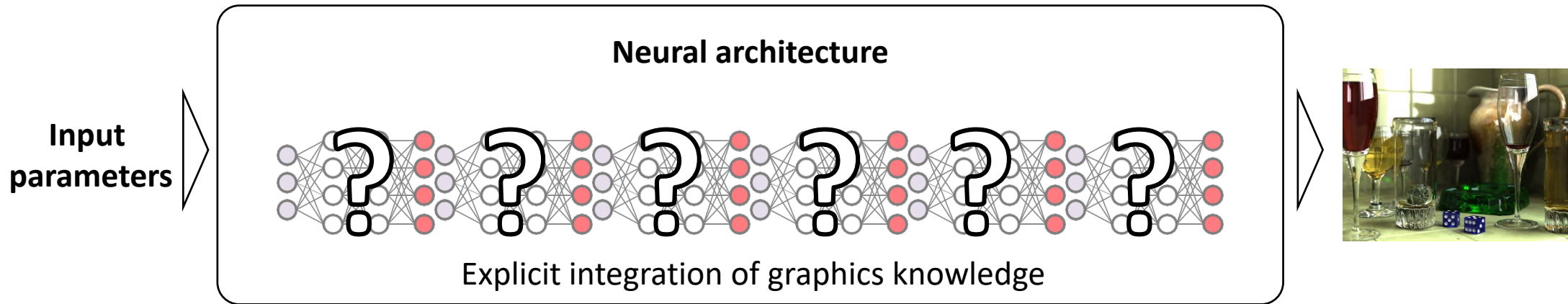


Explicit integration of scene elements and models:

- Geometry: Triangular meshes, primitives (cubes, planes, spheres), voxels, point clouds
- Lights: Parametric primitives (point, area, sphere), environment maps, triangular meshes
- Surface reflectance: Analytical models, measured data, textured

-
-

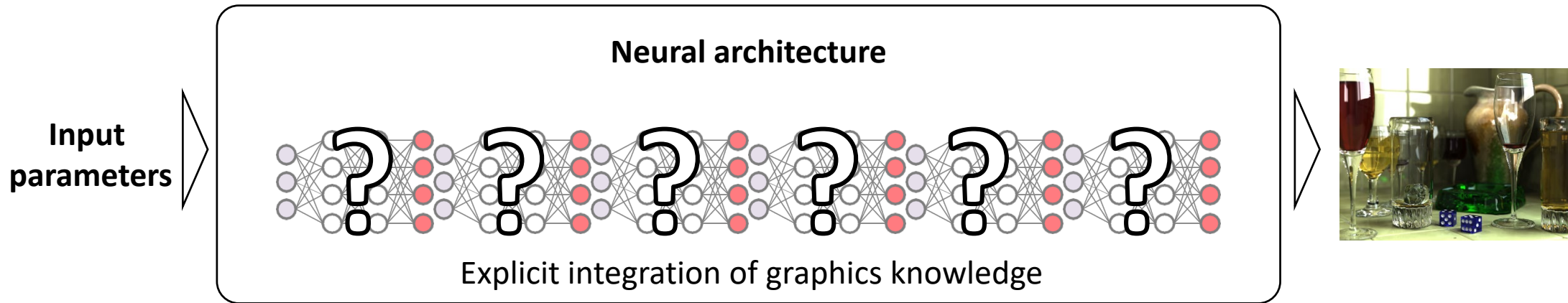
Neural rendering: Integrating CG modules



Explicit integration of scene elements and models:

- Geometry: Triangular meshes, primitives (cubes, planes, spheres), voxels, point clouds
- Lights: Parametric primitives (point, area, sphere), environment maps, triangular meshes
- Surface reflectance: Analytical models, measured data, textured
- Cameras: Pinhole (i.e. perspective projection), lens-based models
-

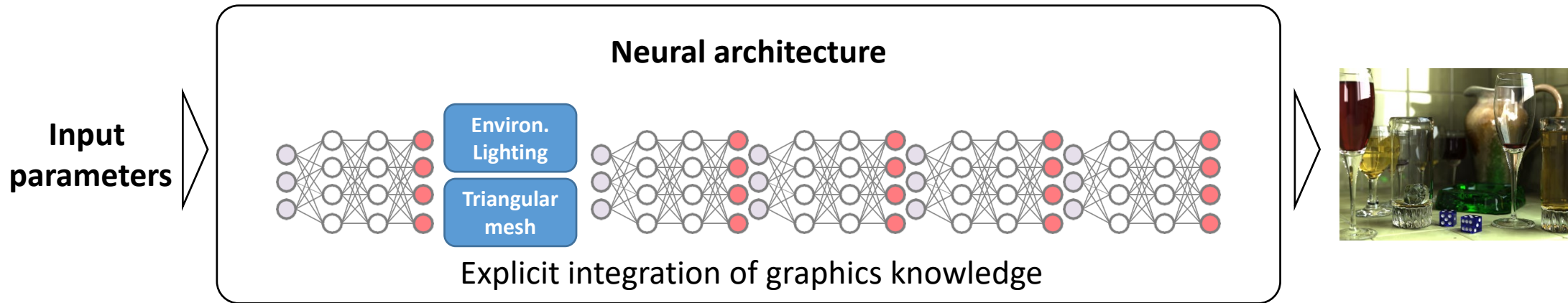
Neural rendering: Integrating CG modules



Explicit integration of scene elements and models:

- Geometry: Triangular meshes, primitives (cubes, planes, spheres), voxels, point clouds
- Lights: Parametric primitives (point, area, sphere), environment maps, triangular meshes
- Surface reflectance: Analytical models, measured data, textured
- Cameras: Pinhole (i.e. perspective projection), lens-based models
- **Rendering methods: Path tracing, ray marching, rasterization**

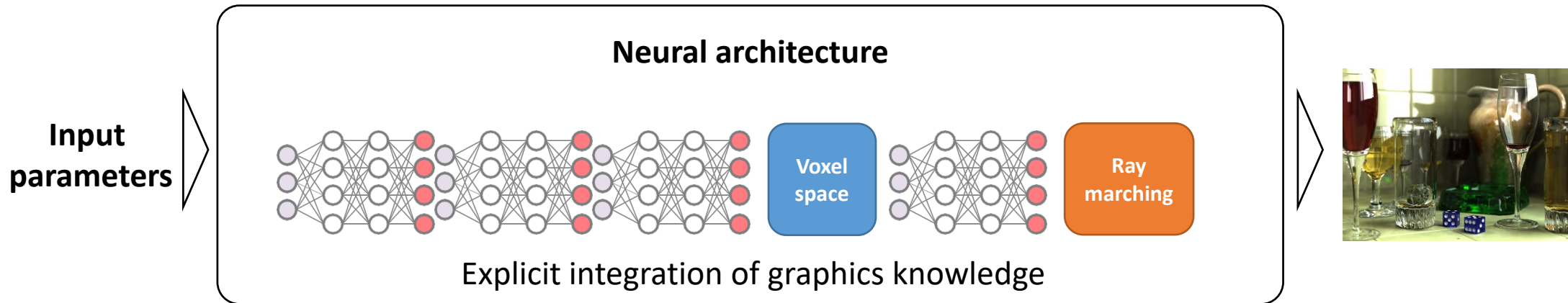
Neural rendering: Integrating CG modules



Explicit integration of scene elements and models:

- Geometry: **Triangular meshes**, primitives (cubes, planes, spheres), voxels, point clouds
- Lights: Parametric primitives (point, area, sphere), **environment maps**, triangular meshes
- Surface reflectance: Analytical models, measured data, textured
- Cameras: Pinhole (i.e. perspective projection), lens-based models
- Rendering methods: Path tracing, ray marching, rasterization

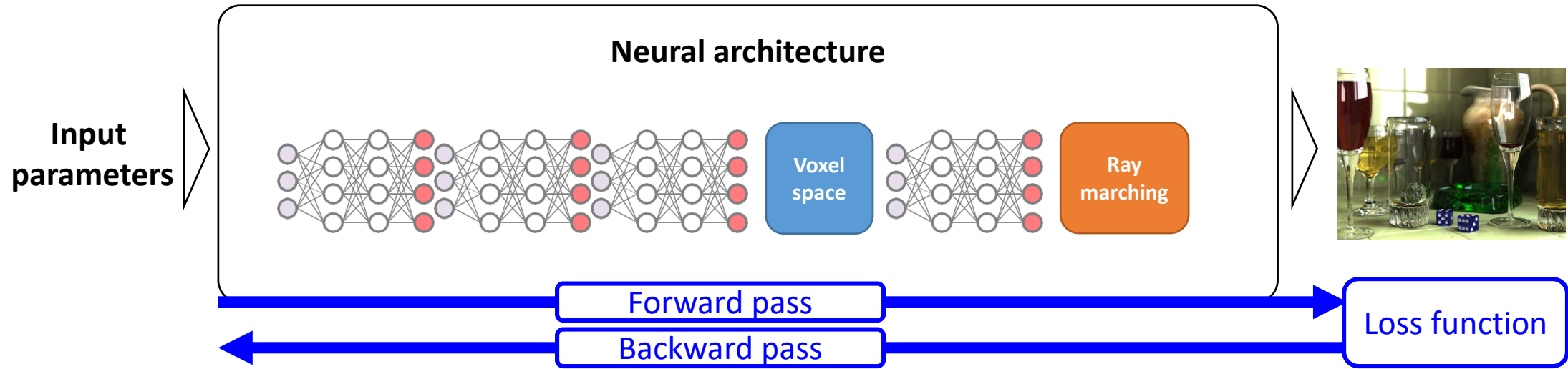
Neural rendering: Integrating CG modules



Explicit integration of scene elements and models:

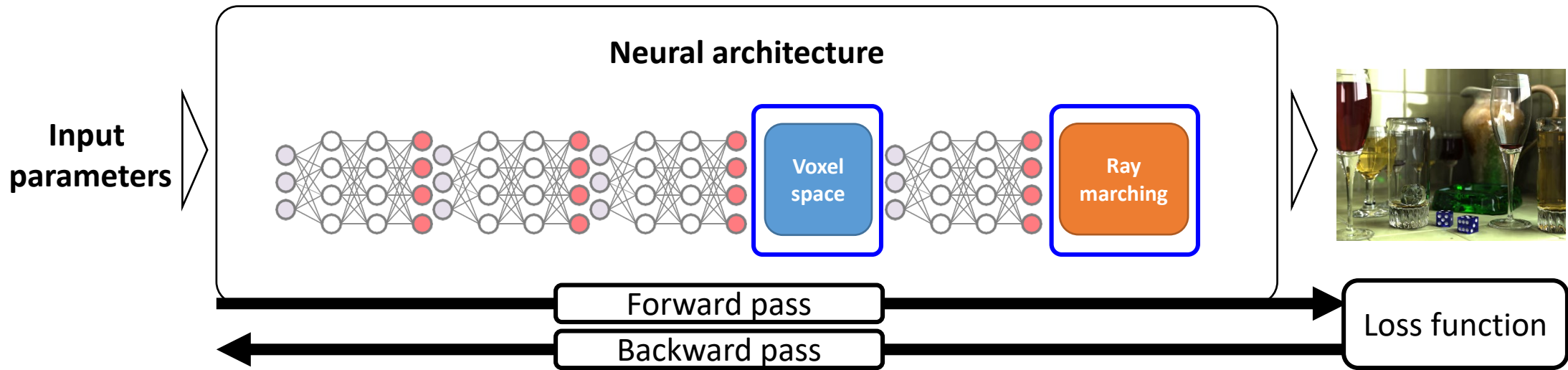
- Geometry: Triangular meshes, primitives (cubes, planes, spheres), **voxels**, point clouds
- Lights: Parametric primitives (point, area, sphere), environment maps, triangular meshes
- Surface reflectance: Analytical models, measured data, textured
- Cameras: Pinhole (i.e. perspective projection), lens-based models
- Rendering methods: Path tracing, **ray marching**, rasterization

Neural rendering: Integrating CG modules



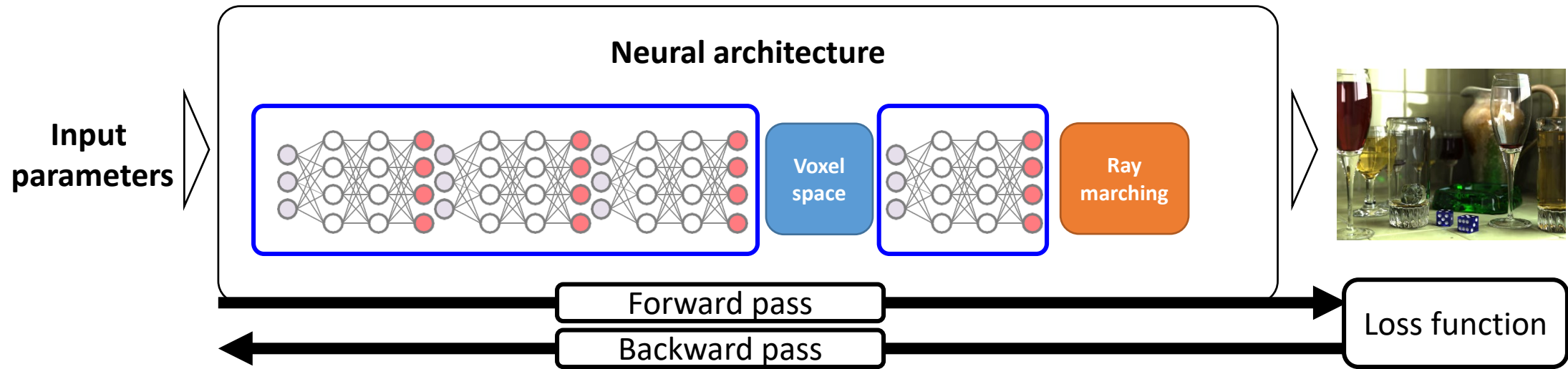
- **End-to-end training** → Backpropagation

Neural rendering: Integrating CG modules



- **End-to-end** training → Backpropagation
- CG modules often need to be **differentiable**. Exception: Transformations to input

Neural rendering: Integrating CG modules



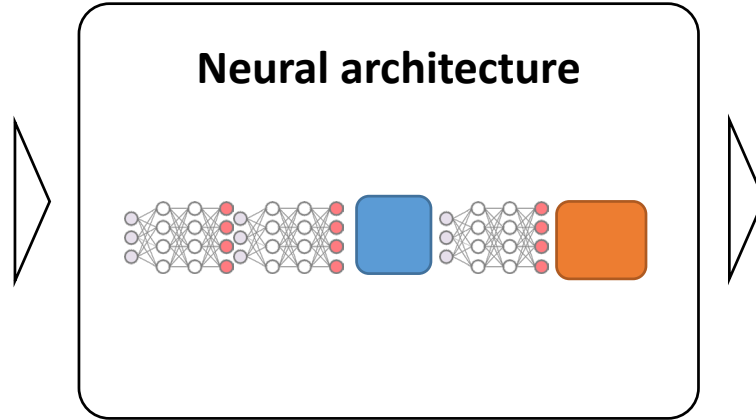
- **End-to-end** training → Backpropagation
- CG modules often need to be **differentiable**. Exception: Transformations to input
- Any other physical behavior is **learned implicitly** by network parameters.

Neural rendering: Controllability

Input parameters



Neural architecture



CONTROLLABILITY

What parameters can we control to generate content? Implicit or explicit control?

IMPLICIT vs. EXPLICIT modeling

What elements are explicitly defined from physics or are implicitly learned from data?

APPLICATION SCOPE

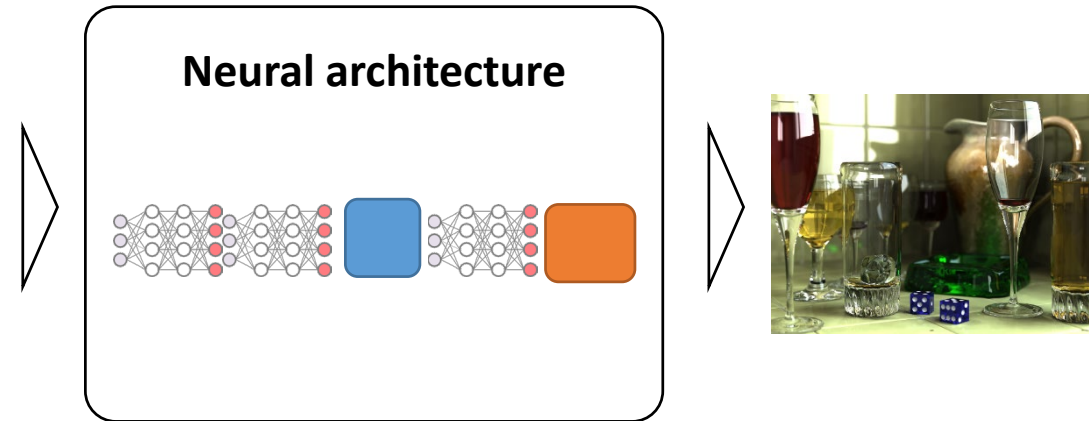
What is our target application? Single or multiple outputs?

Neural rendering: Controllability

Input parameters

Explicit control:

- Scene illumination
- Camera viewpoint
- Semantics (e.g. facial pose, expression)
- Etc.



Source of explicit parameters:

- Synthetic data: Trivial, explicitly defined
- Captured data: Controlled setups, inverse rendering

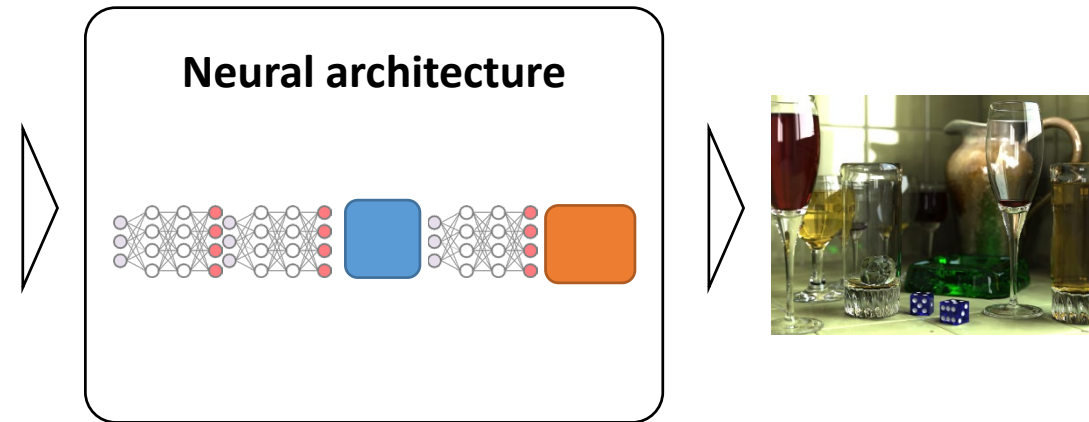


Neural rendering: Controllability

Input parameters

Implicit control:

- Representative example of goal
- Transference, retargeting
- Etc.



Source of implicit parameters:

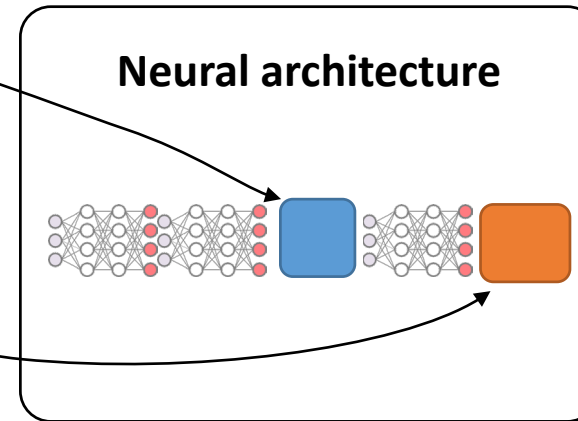
- Image, video examples. Weaker annotation.
- Require less supervision.



Neural rendering: Controllability

- Input parameters**
- Explicit control:
- Scene illumination
 - Camera viewpoint
 - Semantics (e.g. facial pose, expression)

- Implicit control:
- Representative example of goal
 - Transference, retargeting



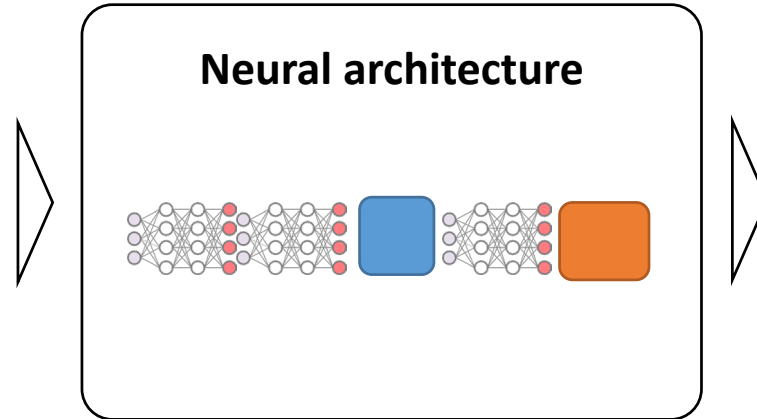
Not only to the first layer!

May use them as input at any training stage, including the CG modules

Neural rendering

Input parameters

- Explicit (light, camera)
- Implicit (annot. examples)



CONTROLLABILITY

What parameters can we control to generate content? Implicit or explicit control?

IMPLICIT vs. EXPLICIT modeling

What elements are explicitly defined from physics or are implicitly learned from data?

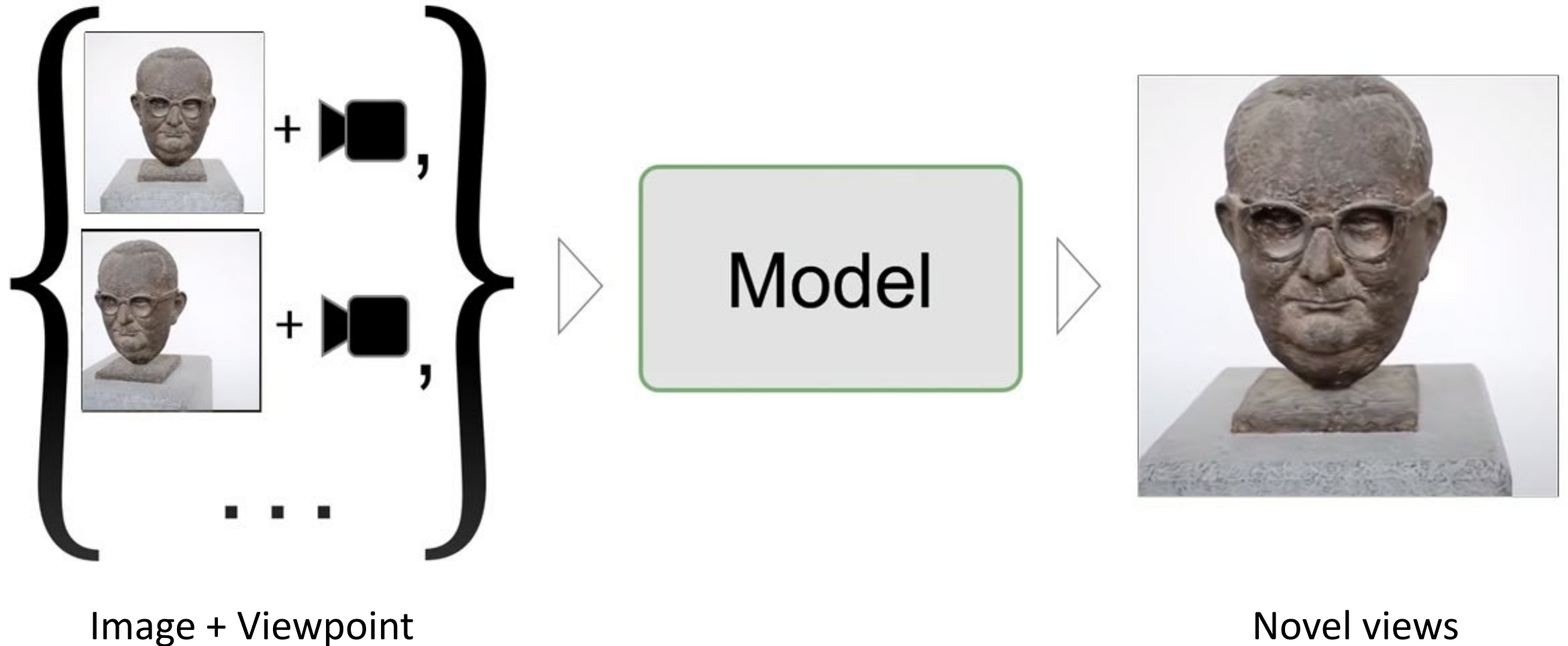
APPLICATION SCOPE

What is our target application? Single or multiple outputs?

Applications: Novel view synthesis

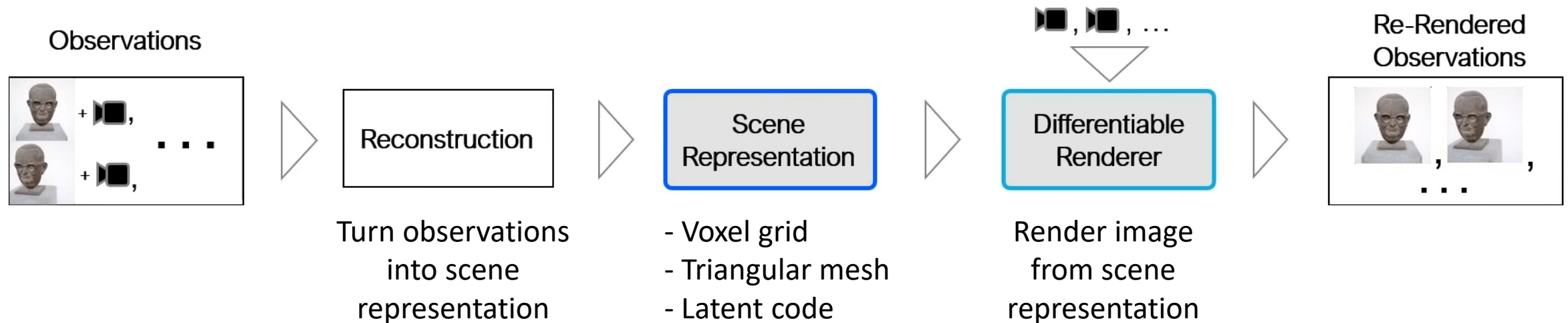
Applications: Novel view synthesis

- Goal: Render novel views from sparse set of observations



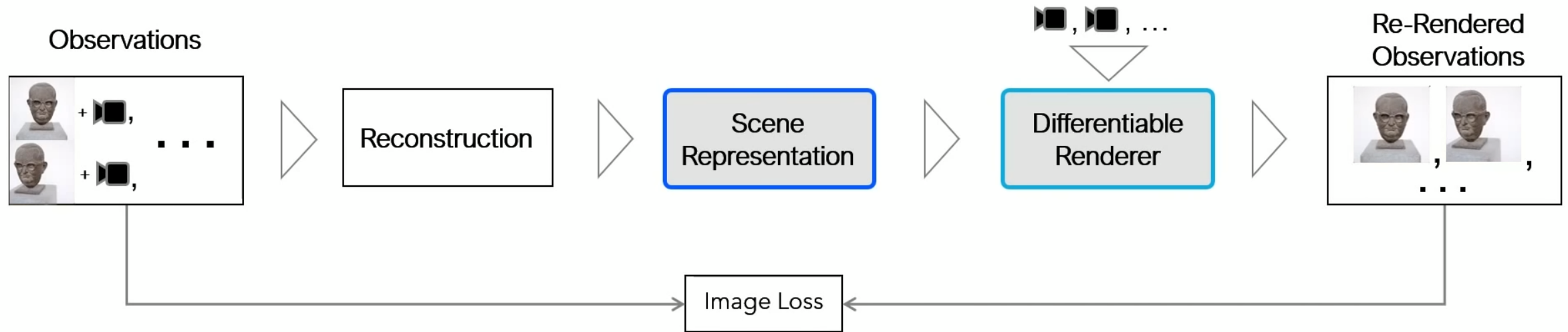
Applications: Novel view synthesis

- Goal: Render novel views from sparse set of observations



Applications: Novel view synthesis

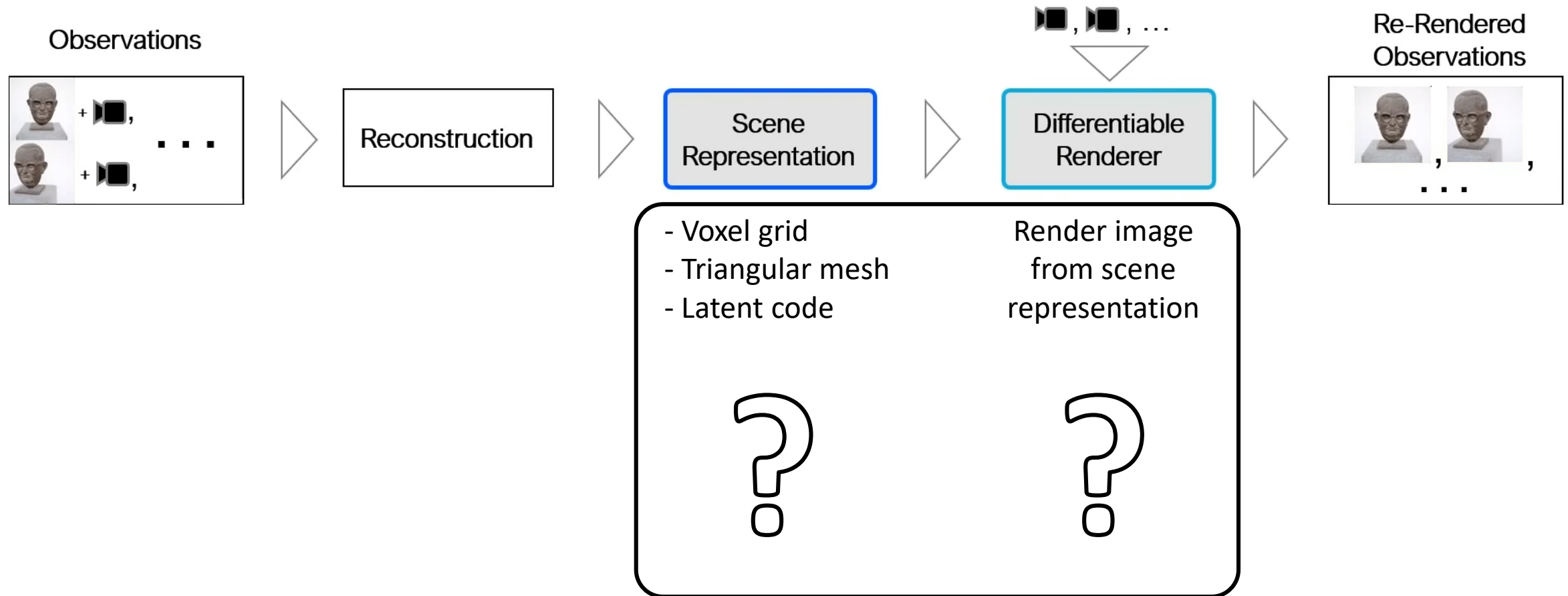
- Goal: Render novel views from sparse set of observations



Train on images

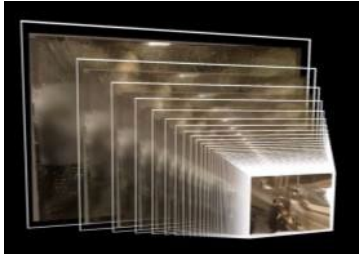
Applications: Novel view synthesis

- Goal: Render novel views from sparse set of observations

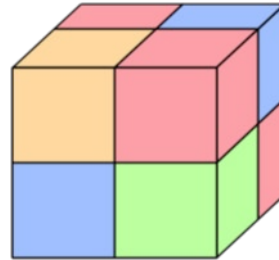


Applications: Novel view synthesis

Scene Representation



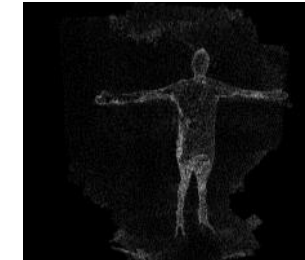
Multi-Plane Images



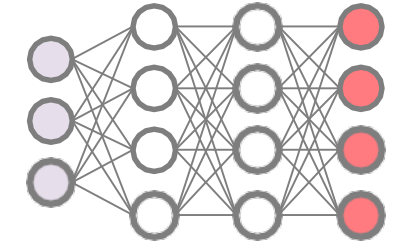
Voxelgrids



Image-based



Point Clouds



Implicit Function

Renderer

(Alpha) compositing

Volumetric
Ray-based

Rasterization

Splatting

Sphere-Tracing
Volumetric

Pros

Fast rendering
High quality
Generalizes

"True 3D"
High quality

High quality

High quality

High quality
May generalize!

Cons

Only 2.5D
Size

No reconstruction
priors
Memory $O(n^3)$

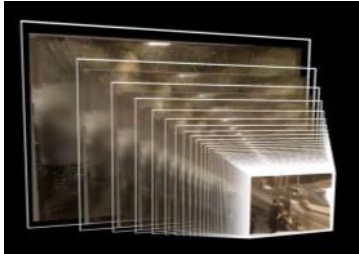
Requires good SFM
No compact
representation

Requires good SFM

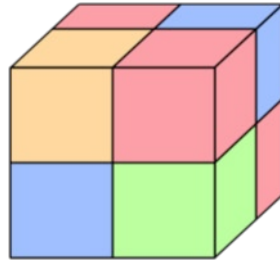
Expensive rendering,
training

Applications: Novel view synthesis

Scene Representation



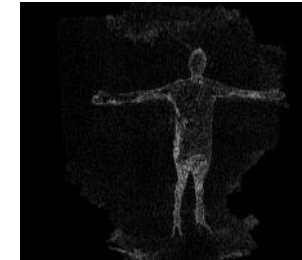
Multi-Plane Images



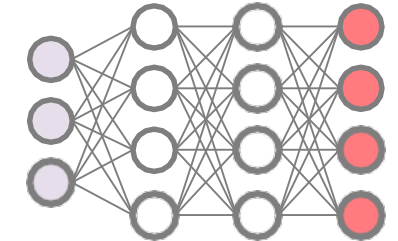
Voxelgrids



Image-based



Point Clouds



Implicit Function

Renderer

(Alpha) compositing

Volumetric
Ray-based

Rasterization

Splatting

Sphere-Tracing
Volumetric

Pros

Fast rendering
High quality
Generalizes

"True 3D"
High quality

High quality

High quality

High quality
May generalize!

Cons

Only 2.5D
Size

No reconstruction
priors
Memory $O(n^3)$

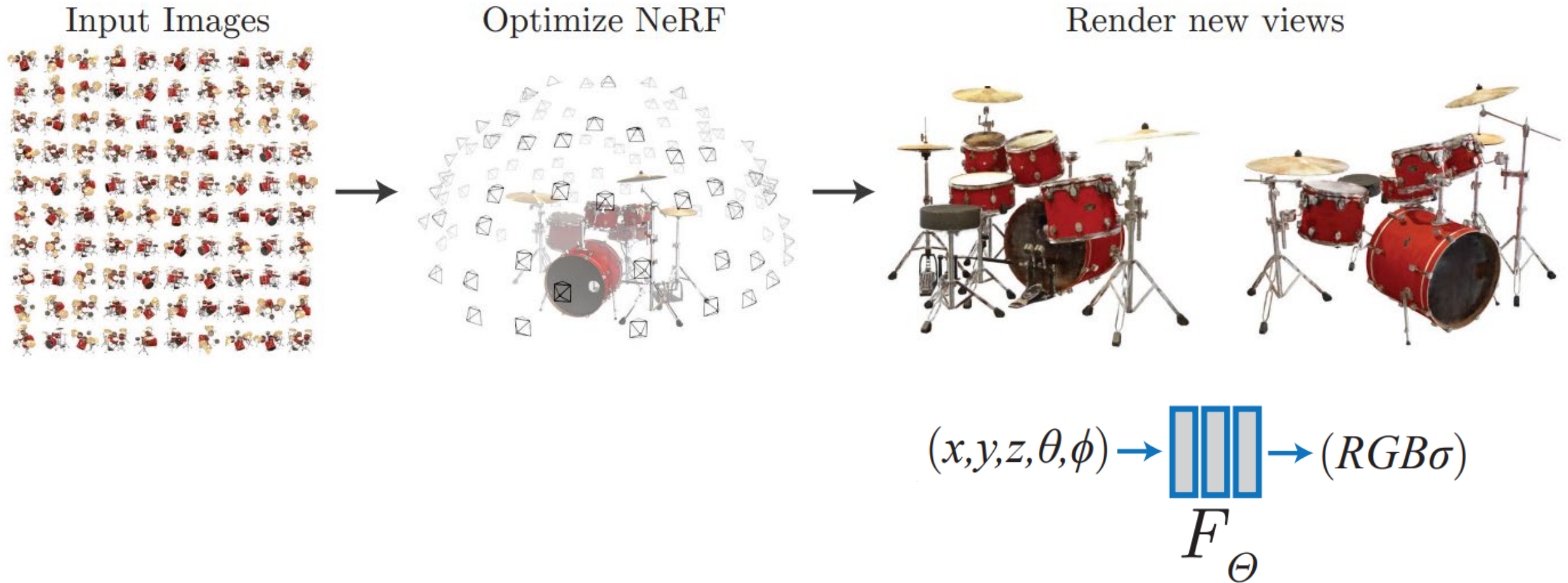
Requires good SFM
No compact
representation

Requires good SFM

Expensive rendering,
training

Applications: Novel view synthesis

NeRF: Neural Radiance Fields [Mildenhall et al. 2020]

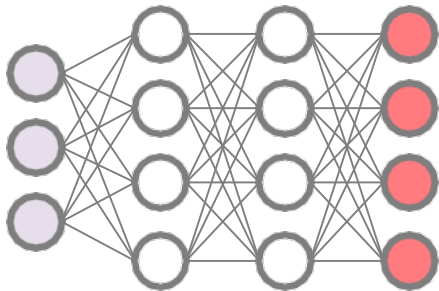


Applications: Novel view synthesis

NeRF: Neural Radiance Fields [Mildenhall et al. 2020]

Scene
Representation

ReLU MLP +
Positional Encoding
View Direction

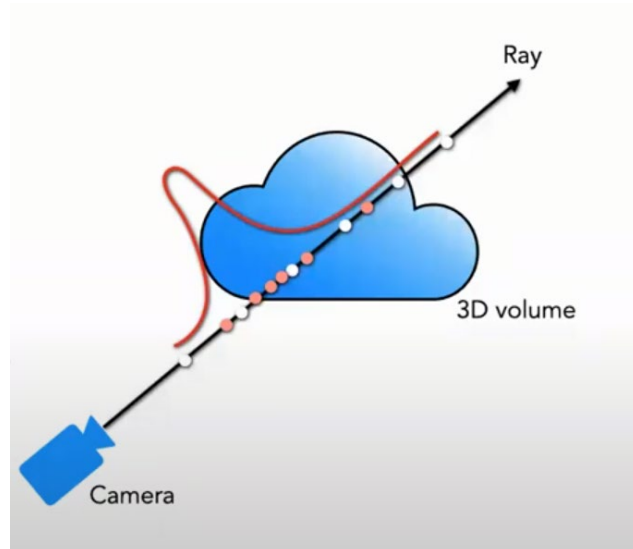


$$\mathbb{R}^5 \rightarrow \mathbb{R}^4$$

$$(x, y, z, \theta, \phi) \rightarrow (r, g, b, \sigma)$$

Renderer

Volumetric,
stratified sampling



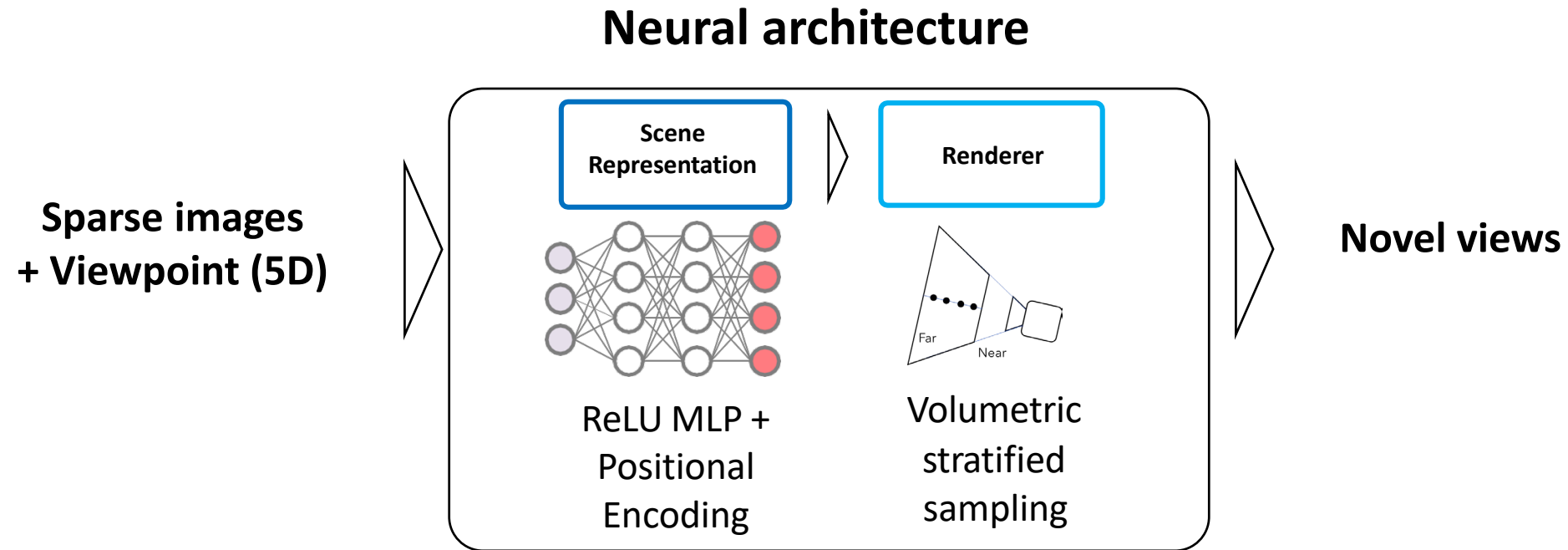
Generalization

Overfitting to a
single scene.



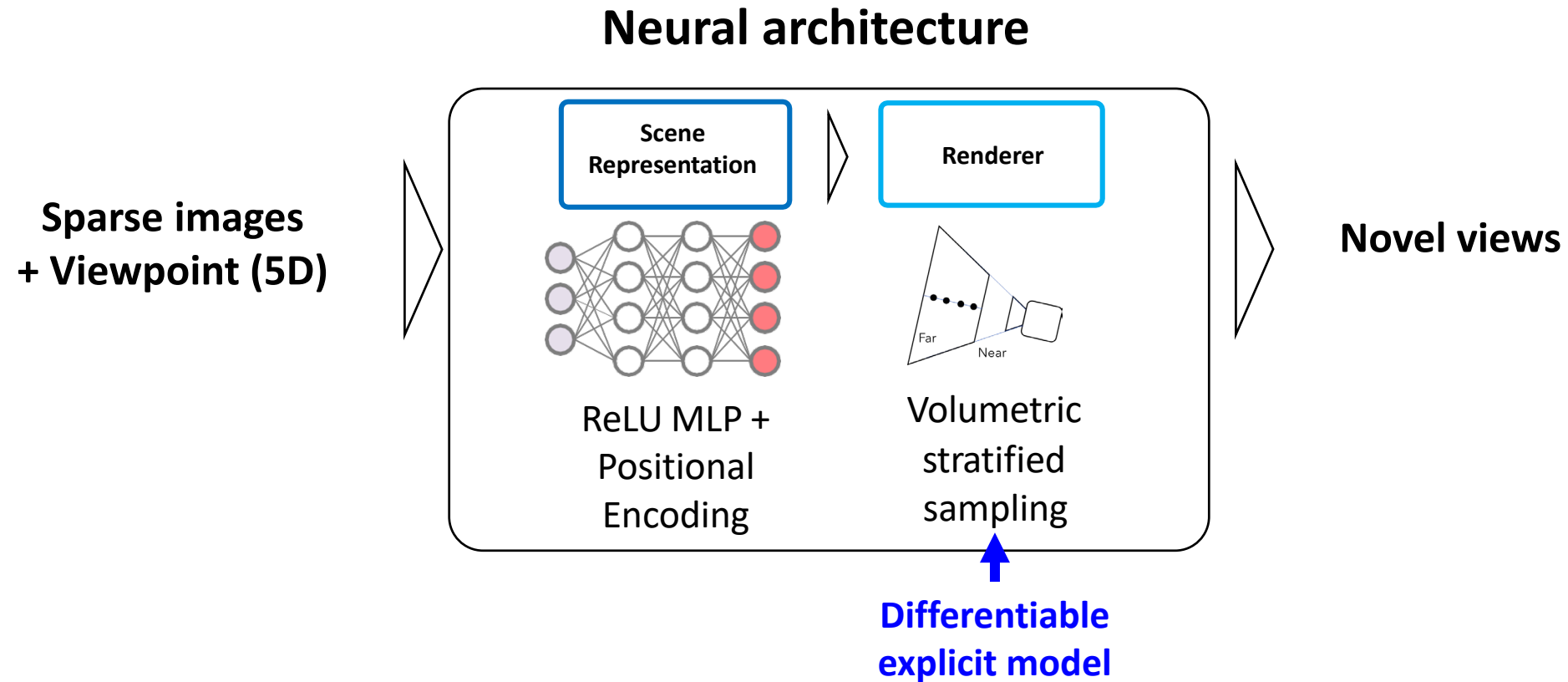
Applications: Novel view synthesis

NeRF: Neural Radiance Fields [Mildenhall et al. 2020]



Applications: Novel view synthesis

NeRF: Neural Radiance Fields [Mildenhall et al. 2020]



Applications: Novel view synthesis

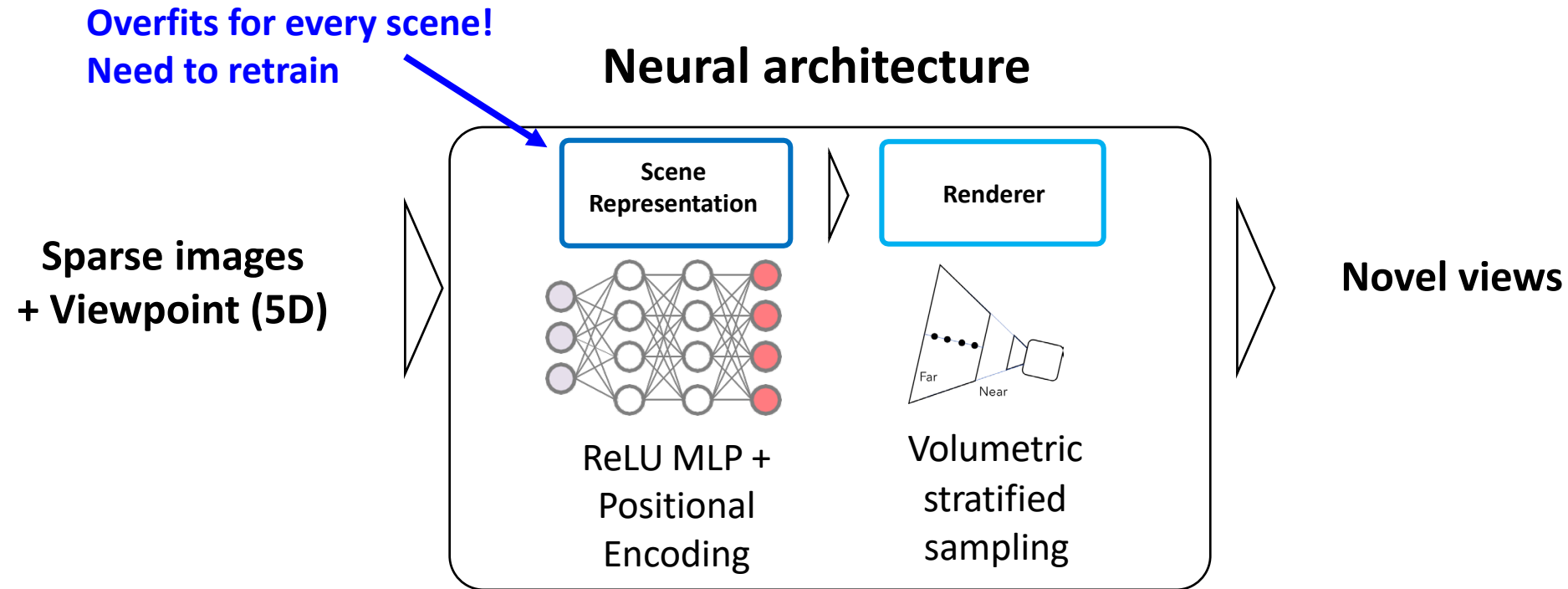
NeRF: Neural Radiance Fields [Mildenhall et al. 2020]

[Source](#)



Applications: Novel view synthesis

NeRF: Neural Radiance Fields [Mildenhall et al. 2020]

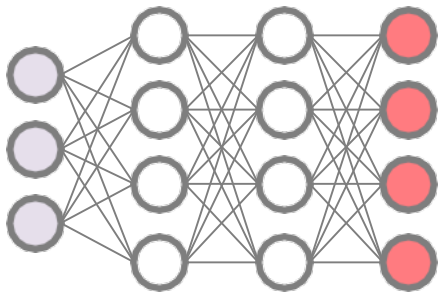


Applications: Novel view synthesis

Scene Representation Networks [Sitzmann et al., Neurips 2019]

Scene
Representation

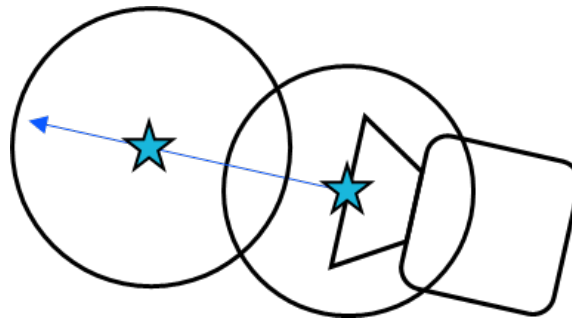
ReLU MLP



$$\mathbb{R}^3 \rightarrow \mathbb{R}^n$$

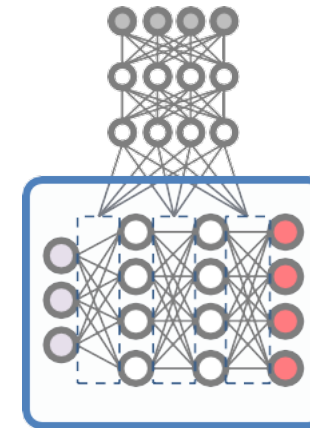
Renderer

Generalized (learned)
sphere-tracing



Generalization

Hypernetwork

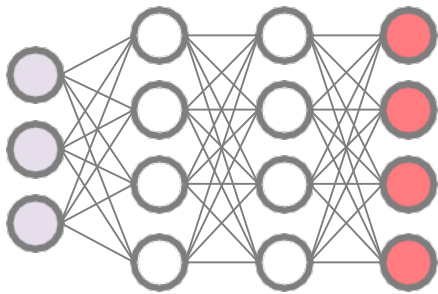


Applications: Novel view synthesis

Scene Representation Networks [Sitzmann et al., Neurips 2019]

Scene
Representation

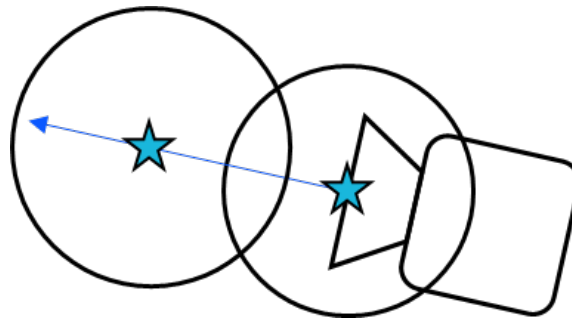
ReLU MLP



$$\mathbb{R}^3 \rightarrow \mathbb{R}^n$$

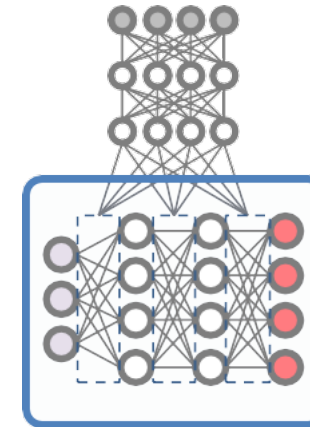
Renderer

Generalized (learned)
sphere-tracing



Generalization

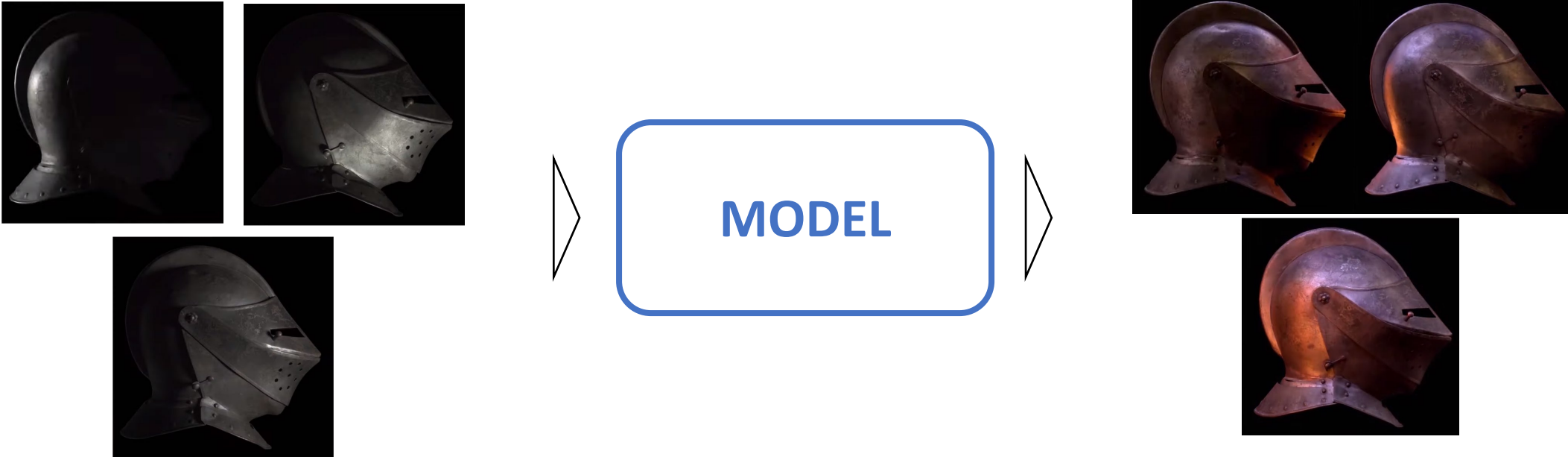
Hypernetwork



Applications: Scene relighting

Applications: Scene relighting

- Goal: Modifying the illumination conditions of a scene

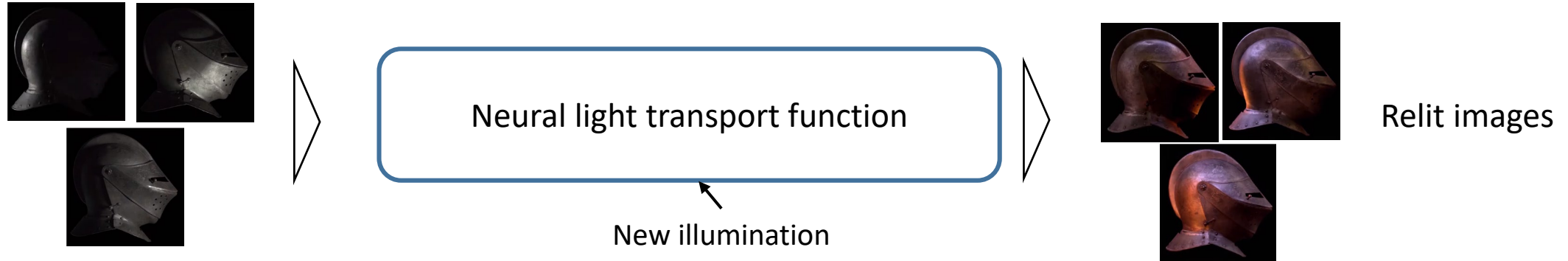


Sparse observations under
Known different illumination

Novel illumination

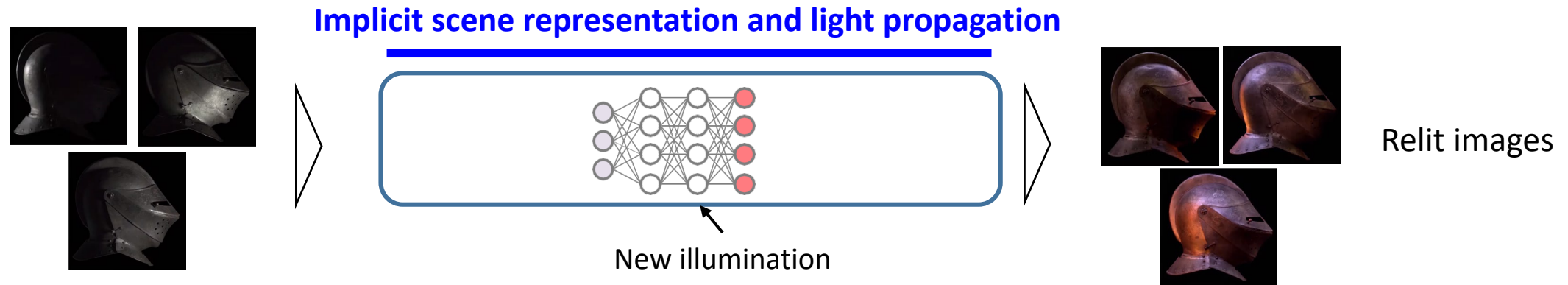
Applications: Scene relighting

- Goal: Modifying the illumination conditions of a scene



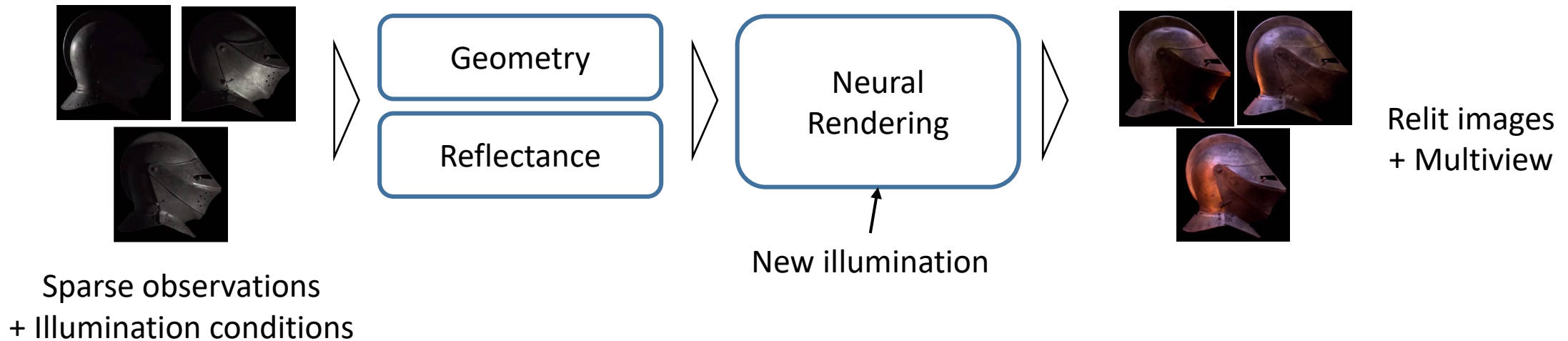
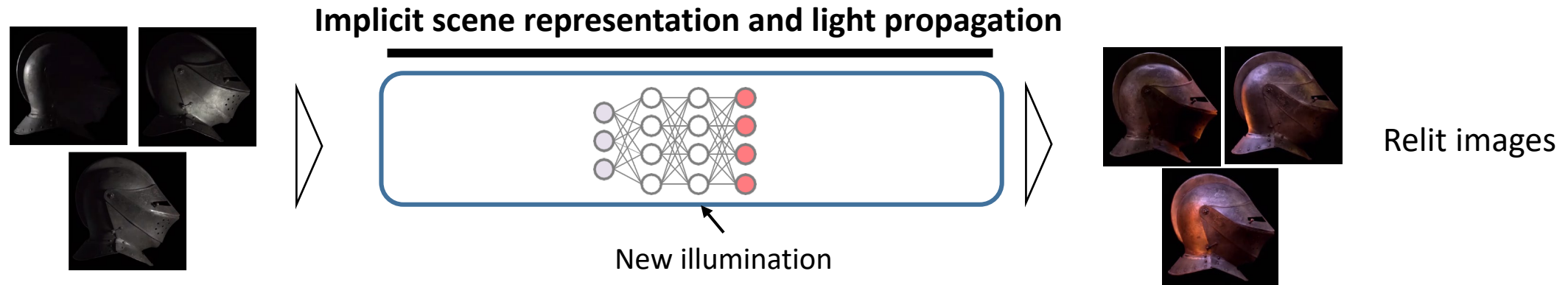
Applications: Scene relighting

- Goal: Modifying the illumination conditions of a scene



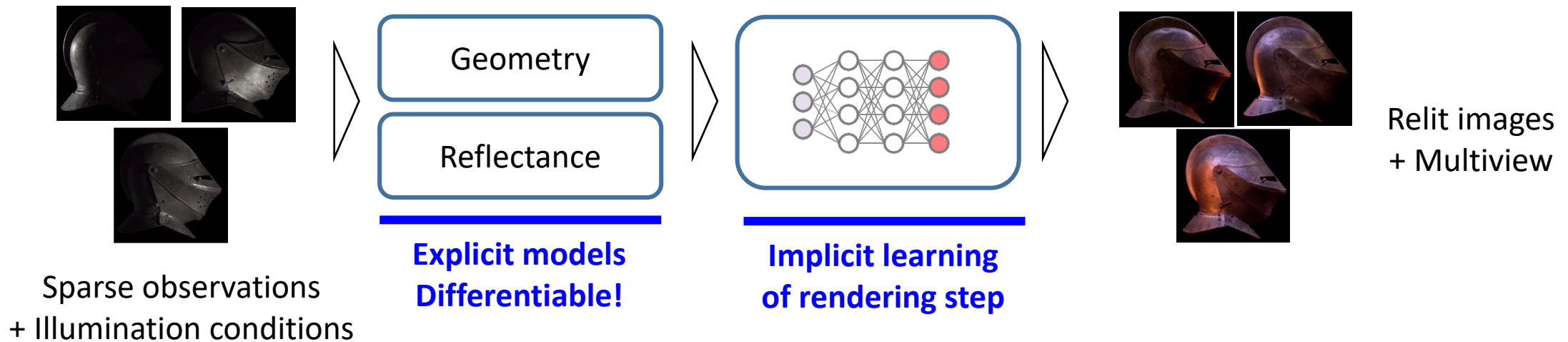
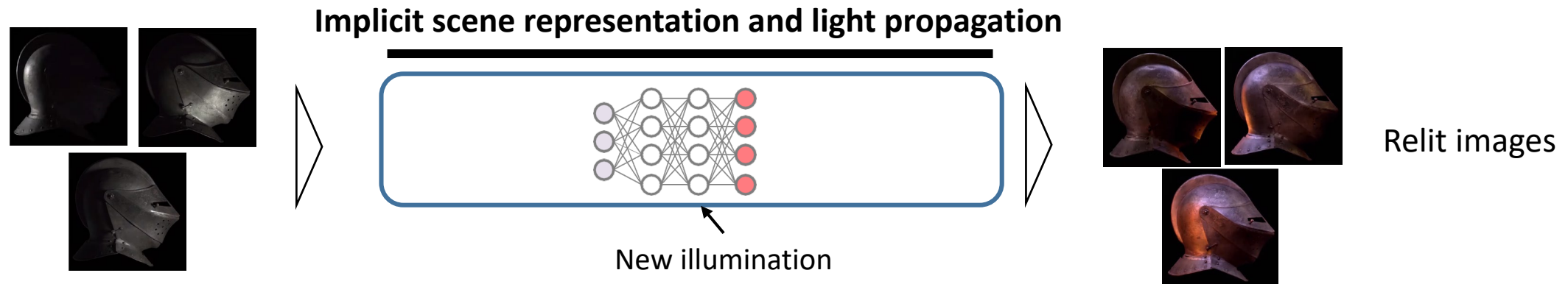
Applications: Scene relighting

- Goal: Modifying the illumination conditions of a scene



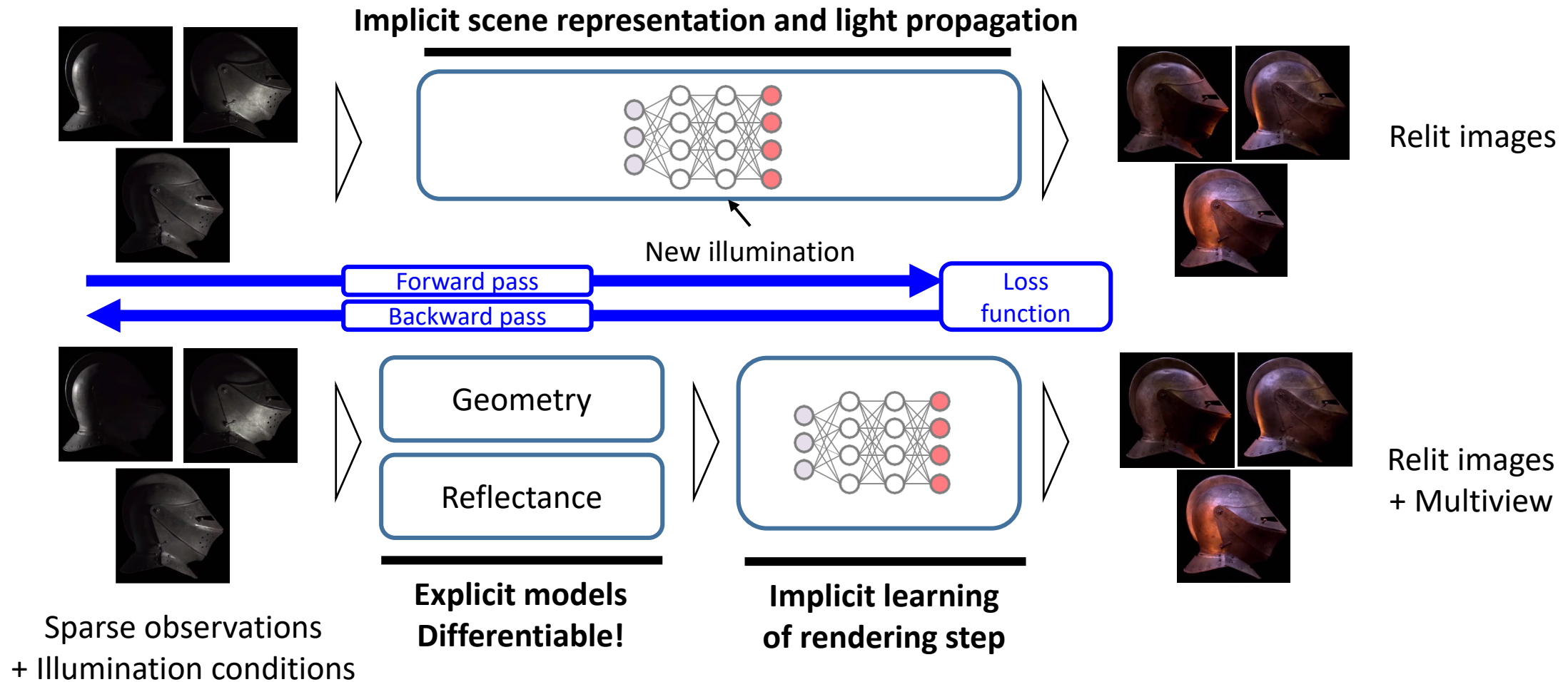
Applications: Scene relighting

- Goal: Modifying the illumination conditions of a scene



Applications: Scene relighting

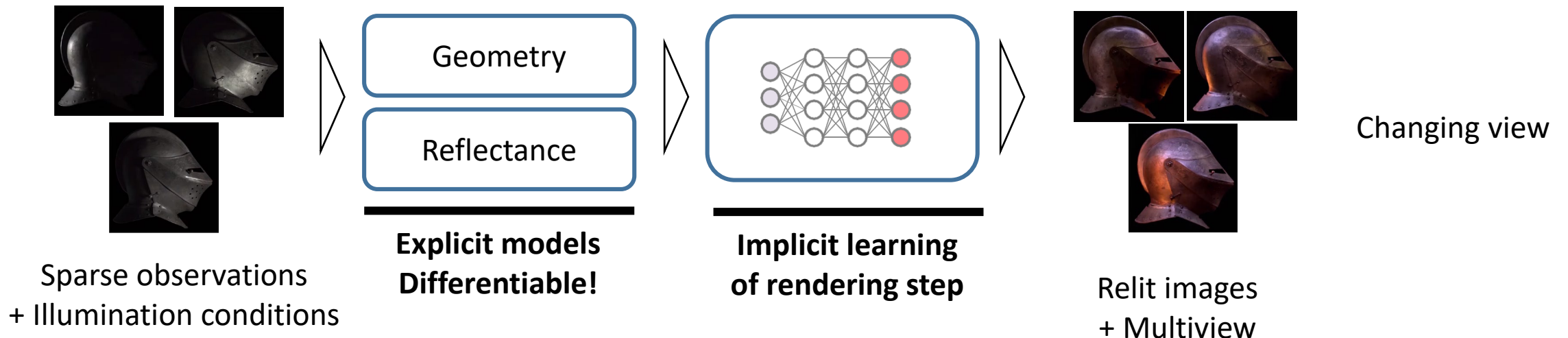
- Goal: Modifying the illumination conditions of a scene



Applications: Scene relighting

- Goal: Modifying the illumination conditions of a scene

IMAGE-BASED RELIGHTING



Applications: Scene relighting

- Goal: Modifying the illumination conditions of a scene

IMAGE-BASED RELIGHTING



RECONSTRUCTION-BASED RELIGHTING



Applications: Scene relighting

- Multi-view Relighting using a Geometry-Aware Network [Philip et al. 2019]



Applications: Scene relighting

- Multi-view Relighting using a Geometry-Aware Network [Philip et al. 2019]



(a) input photo



(b) same scene, 3h30 later (real photo)



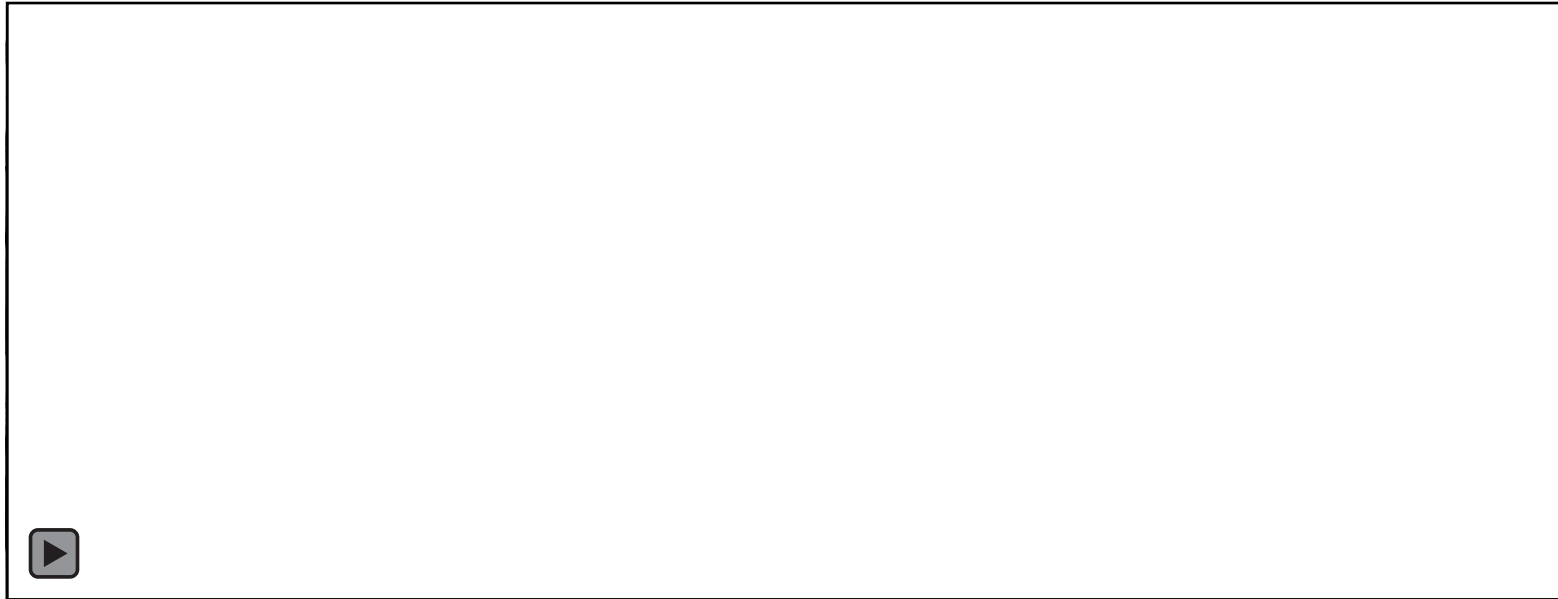
(c) our output

Applications: Novel view synthesis

NeRF: Neural Radiance Fields [Mildenhall et al. 2020]

➔ **Implicitly encodes light changes**

[Source](#)



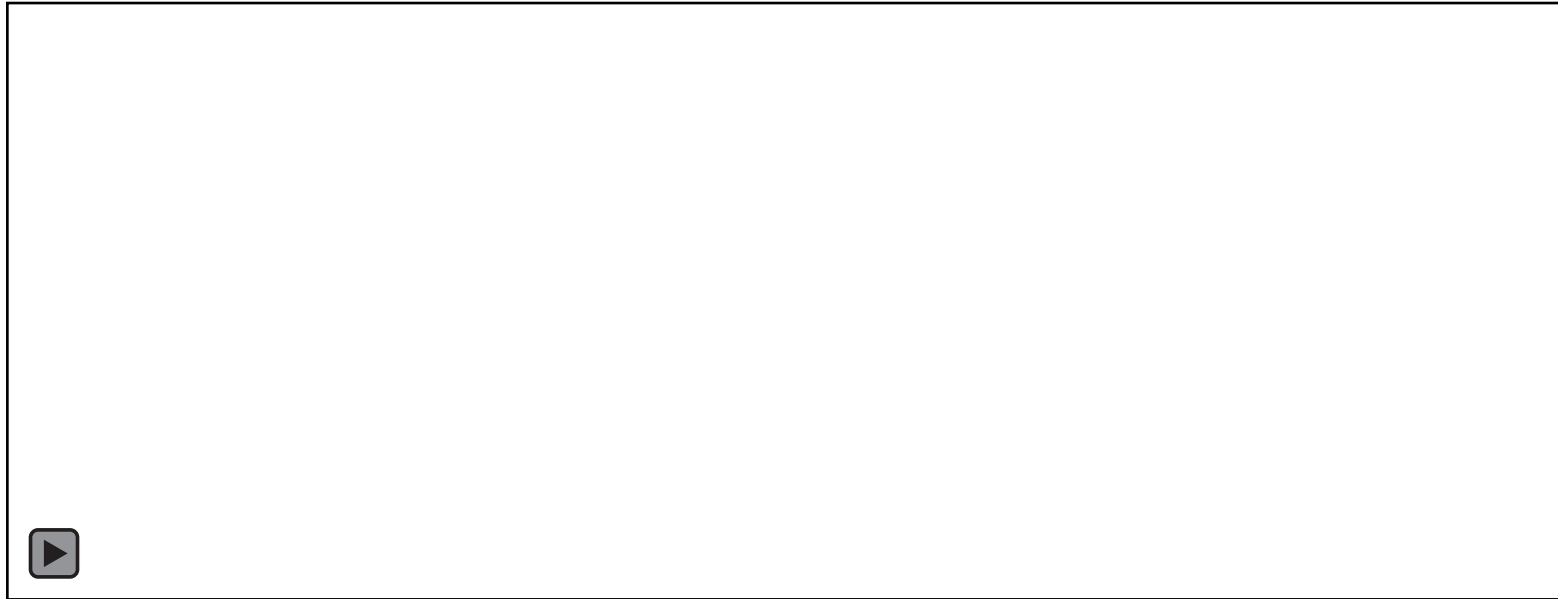
Fixed viewpoint
Changing query direction

Applications: Novel view synthesis

NeRF: Neural Radiance Fields [Mildenhall et al. 2020]

➔ **Implicitly encodes light changes**

[Source](#)



Fixed viewpoint
Changing query direction

Scene relighting - References

- [Sun et al. 2019] Single-image portrait relighting.
- [Xu et al. 2018] Deep image-based relighting from optimal sparse samples.
- [Xu et al. 2019] Deep view synthesis from sparse photometric images
- [Kanamori et al. 2019] Relighting Humans: Occlusion-Aware Inverse Rendering for Full-Body Human Images
- [Mekas et al. 2019] Deep reflectance fields: High-quality facial reflectance field inference from color gradient illumination.
- [Nestmeyer et al. 2020] Learning Physics-guided Face Relighting under Directional Light
- [Zhou et al. 2019] Deep single image portrait relighting.
- [Chen et al. 2020] A Neural Rendering Framework for Free-Viewpoint Relighting